

Multiagent Systems

Study Text for the Final State Examination

Final State Examination in Artificial Intelligence, MFF UK

July 2026

This text covers the examination topic “Multiagent Systems” of the master’s final state examination in Artificial Intelligence at the Faculty of Mathematics and Physics, Charles University. It is based on the lecture course NAIL106 *Multiagent Systems* (MFF UK) and on the textbooks M. Wooldridge: *An Introduction to MultiAgent Systems* (2nd ed., 2009), Y. Shoham, K. Leyton-Brown: *Multiagent Systems* (2009) and S. Russell, P. Norvig: *Artificial Intelligence: A Modern Approach*. This is an English translation of the Czech study text *Multiagentní systémy*; terminology follows the standard usage of the sources listed above.

Mapping of the examination topic to chapters

Item of the official examination topic	Chapter in this text
Architecture of an autonomous agent; action selection mechanism	1 Introduction, 2 Architecture of an autonomous agent
Methods for controlling agents; symbolic reactive planning	3 Deliberative architectures, 4.1–4.3
Connectionist reactive planning	4.4 Agent network architecture (Maes)
Hybrid approaches	4.6 Hybrid architectures, 4.7 IDA
The pathfinding problem	5 Pathfinding (A*, D* Lite, MAPF)
Communication and knowledge in MAS, ontologies	6 Knowledge and ontologies
Speech acts, FIPA-ACL, protocols	7 Agent communication
Distributed problem solving, cooperation	8 Distributed problem solving (CNET, BBS, DCOP)
Nash equilibria, Pareto efficiency	9 Agent interaction and game theory (incl. coalitional games)
(social choice as a complement to preference aggregation)	10 Social choice and voting
Resource allocation, auctions	11 Resource allocation, auctions, negotiation (+ mechanism design)
Methods for agent learning; reinforcement learning	12 Agent learning
Design methodologies, MAS languages and environments	13 Methodologies, languages, and platforms

Contents

Mapping of the examination topic to chapters	1
1 Introduction: agent and multiagent system	4
1.1 Motivation	4
1.2 What an agent is	4
1.3 Properties of an intelligent agent	5
1.4 Intentional systems and the intentional stance	5
1.5 Exam summary	5
2 Architecture of an autonomous agent and the action selection mechanism	6
2.1 The environment and its properties	6
2.2 An abstract agent architecture	6
2.3 Purely reactive agents and agents with internal state	7

2.4	How to tell an agent what to do: utility	8
2.5	Predicate task specification	9
2.6	The action selection mechanism — an overview	9
2.7	Exam summary	10
3	Deliberative architectures: symbolic reasoning, BDI, PRS	10
3.1	The classical AI approach and the deductive agent	10
3.2	Practical reasoning: the BDI architecture	12
3.3	Planning: STRIPS	12
3.4	Implementing a BDI agent	14
3.5	Agent commitment	14
3.6	The Procedural Reasoning System (PRS)	15
3.7	Exam summary	16
4	Reactive and hybrid architectures	16
4.1	The reactive approach and its premises	16
4.2	Symbolic reactive planning: the subsumption architecture	17
4.3	Connectionist reactive planning: the agent network architecture	18
4.4	Limitations of reactive architectures	19
4.5	Hybrid architectures	19
4.6	Complex architectures: IDA and the global workspace	20
4.7	Comparison of architectures	21
4.8	Exam summary	21
5	The pathfinding problem	22
5.1	Formulation and context	22
5.2	A* and related algorithms	22
5.3	Pathfinding in a dynamic environment	23
5.4	Multi-agent path finding (MAPF)	24
5.5	Exam summary	26
6	Knowledge and ontologies	26
6.1	From an agent to a multiagent system	26
6.2	Reasoning about knowledge: epistemic logic and common knowledge	26
6.3	What an ontology is	27
6.4	Ontology languages	27
6.5	Exam summary	29
7	Agent communication: speech acts, ACL, protocols	29
7.1	Why agent communication is different	29
7.2	Speech act theory	29
7.3	KQML and FIPA-ACL	31
7.4	Interaction protocols	32
7.5	Exam summary	33
8	Distributed problem solving and cooperation	33
8.1	Coherence and coordination	33
8.2	Task sharing and result sharing	34
8.3	The Contract Net	34
8.4	Blackboard systems	34
8.5	Distributed constraints (DisCSP and DCOP)	35
8.6	Joint intentions and social laws	36
8.7	Agent interaction in the environment	36
8.8	Exam summary	36

9	Agent interaction and game theory	37
9.1	The selfish agent	37
9.2	Normal-form games	37
9.3	Classical games	38
9.4	Cooperative (coalitional) games	40
9.5	Exam summary	41
10	Social choice and voting	41
10.1	Preference aggregation	41
10.2	Voting systems	42
10.3	A worked example: one preference schedule, four winners	43
10.4	Properties of voting systems	44
10.5	Exam summary	44
11	Resource allocation, auctions, and negotiation	45
11.1	Auctions as a mechanism of resource allocation	45
11.2	Mechanism design	45
11.3	The four basic types of auction	46
11.4	Combinatorial auctions and VCG	47
11.5	Further types of auction and resource allocation	48
11.6	Negotiation	49
11.7	Exam summary	49
12	Agent learning and reinforcement learning	50
12.1	Why and how agents learn	50
12.2	The Markov decision process	50
12.3	Q-learning and SARSA	51
12.4	Policy-based methods	52
12.5	Multiagent reinforcement learning	53
12.6	Exam summary	54
13	Design methodologies, languages, and MAS platforms	55
13.1	Agent-oriented software engineering	55
13.2	The FIPA standard and the architecture of a platform	56
13.3	Languages and platforms	56
13.4	Exam summary	57
	In closing: how to approach the exam	58

1 Introduction: agent and multiagent system

1.1 Motivation

Multiagent systems (MAS) emerged as an independent area of computer science in the 1990s in response to several concurrent trends: the *ubiquity* of computing technology, *connectivity and distribution* (computation as interaction), the growing *complexity* of tasks, the *delegation* of tasks to software, and the pressure towards ease of use. The classical paradigm “a program receives an input, computes an output, and terminates” ceases to be adequate the moment software is permanently embedded in an environment that changes independently of it and in which other independent components run alongside it.

MAS can be viewed from four different perspectives, which explains the breadth of the topic:

- **Software engineering.** An agent is the next level of abstraction in the sequence machine code → structured programs → objects → distributed objects (CORBA) → agents. The classic quip: “objects do it for free, agents do it because they want to”.
- **Distributed and parallel systems.** Decades of research have solved mutual exclusion, deadlock, and consensus. The difference is that agents are *autonomous*, coordination mechanisms are not given in advance, and everything has to be resolved at run time.
- **Artificial intelligence.** Traditionally: MAS is a subfield of AI. According to Russell and Norvig the goal of AI is conversely the design of intelligent agents (hence $AI \subseteq MAS$); Etzioni’s pointed version is “MAS is 1 % AI and 99 % computer science”. MAS uses AI techniques (knowledge representation, planning) but emphasises the *social* aspects that classical AI overlooked.
- **Game theory and economics.** A formal apparatus for rational decision making by multiple parties. Mathematical game theory deals mainly with questions of *existence*, whereas MAS deals with the *computational* aspects (complexity, practical usability).
- **Sociology.** A key notion of MAS is *agent societies*, and sociology studies human societies — so the inspiration suggests itself. The same caveat applies here as to the relation between AI and human intelligence: inspiration yes, necessity no (aeroplanes do not flap their wings). In the opposite direction the link is stronger: sociology, economics, and ecology all use agents as a building block of their simulation models — *agent-based modelling* (ABM), see the platforms in chapter 13.

1.2 What an agent is

There is no single agreed definition of an agent — Franklin and Graesser (1996), in the paper *Is it an agent, or just a program?*, collected dozens of mutually incompatible definitions. The crucial point is to distinguish an agent from an ordinary procedure or object. The most frequently cited formulations:

Agent — working definitions

Russell, Norvig (AIMA): An agent is anything that can be viewed as an entity *perceiving* its environment through sensors and *acting* upon that environment through effectors.

Pattie Maes: Autonomous agents are computational systems that inhabit some complex dynamic environment, sense and act autonomously in this environment, and by doing so realise a set of goals or tasks for which they were designed.

Wooldridge, Jennings: An agent is a computer system situated in some environment that is capable of *autonomous action* in that environment in order to meet its delegated goals.

Multiagent system

A multiagent system is a system composed of several agents that *interact* with one another — most often by exchanging messages over a computer network. The agents *need not share a common goal*; they may have their own, even conflicting, interests. This is why it is necessary

to study mechanisms of negotiation, coordination, and dynamic resource allocation.

Autonomy. A human being is fully autonomous, a Java method not at all. An agent lies in between: it should autonomously choose the *way* in which a given problem is to be solved (that is, the choice of subgoals), but not the main goal — that is delegated to it.

1.3 Properties of an intelligent agent

- **Reactivity** — it perceives the environment and is able to respond to changes in it within a reasonable (real) time.
- **Proactiveness** — it has its own goals and actively pursues them, on its own initiative, not merely as a response to a stimulus.
- **Social ability** — it is able to communicate with other agents and with humans.

Achieving *purely* reactive or *purely* goal-oriented behaviour is fairly easy. What is hard is to find the **balance between reactivity and proactiveness**: an agent must not blindly follow a plan that has become pointless, but neither may it constantly reconsider its goals and never get anywhere. This trade-off recurs in various guises throughout the whole text (see in particular commitment strategies in chapter 3 and hybrid architectures in chapter 4).

1.4 Intentional systems and the intentional stance

When we talk about agents, we ascribe *mental states* to them: “the agent believes that...”, “the agent wants...”. Daniel Dennett defines an **intentional system** as an entity whose behaviour can be predicted by ascribing to it properties such as beliefs, desires, and rationality. Intentional systems form a hierarchy: a *first-order* system has beliefs and desires, a *second-order* system has beliefs and desires about beliefs and desires (its own as well as those of others), and so on.

Dennett distinguishes three *stances* towards predicting the behaviour of a system: the *physical* one (the laws of physics suffice — if I throw a stone I need not speak of its desires, mass, velocity, and gravity are enough), the *design stance* (I do not know all the physical laws, but I know the purpose for which the system was designed — I can set an alarm clock without understanding its electronics), and the *intentional* one (I describe the system in terms of beliefs, desires, and rationality).

Shoham gives a provocative example: “The light switch in the room is a very cooperative agent that is willing of its own free will to conduct electric current, but does so only when it believes that we really want it to.” The description is consistent and elegant, yet most people find it absurd — *because for a light switch we have a simpler and equally accurate description*. The intentional stance pays off where a physical or design description does not exist or is impractical (even with the schematic of a computer at hand it is hard to derive why clicking an icon opens a window). **From the point of view of computer science, the intentional stance is above all an abstraction for managing complexity** — and for many programmers that is precisely the main contribution of MAS.

There remains the philosophical question Dennett poses: does the intentionality of entities exist *in itself*, or does it arise only *in the process of interpretation* by an observer? Dennett leans towards the second option — no intentionality without interpretation. For us this has a practical consequence: ascribing beliefs and desires to an agent is *our description*, not a property of the implementation, and it is legitimate exactly to the extent that it helps us predict behaviour.

1.5 Exam summary

- Agent = an autonomous system situated in an environment, which perceives and acts in it in order to meet delegated goals. MAS = a system of interacting agents *without a necessarily shared goal*.

- The key problem in agent design is **action selection**; an agent architecture is a software architecture that realises this selection.
- Three properties of an intelligent agent: reactivity, proactiveness, social ability; the hard part is balancing them.
- Dennett: the intentional system and the three stances (physical, design, intentional); the intentional stance is an abstraction for managing complexity.
- Distinguishing MAS from: distributed systems (autonomy, coordination only at run time), objects (an agent performs an action only if it wants to), classical AI (emphasis on social aspects), game theory (emphasis on computational aspects).

2 Architecture of an autonomous agent and the action selection mechanism

2.1 The environment and its properties

An agent's behaviour is fundamentally determined by the type of environment it finds itself in. The standard classification (Russell, Norvig):

Dichotomy	Meaning
fully / partially observable	The agent's sensors give it the complete state of the environment — or only part of it (hidden information, noise, limited range).
static / dynamic	The environment changes only as a result of the agent's actions — or also otherwise (other agents, natural processes). In a static environment the agent can “think its next move over at leisure”.
deterministic / non-deterministic	Every action has exactly one possible outcome — or several (with a known or unknown probability distribution; in that case we speak of a stochastic environment).
discrete / continuous	A finite number of states and actions — or continuous quantities (position, velocity).

Further commonly listed dichotomies: *episodic* / *sequential* (does today's action affect future rewards?), *single-agent* / *multiagent*, *known* / *unknown* (does the agent know the transition function in advance?).

An unpleasant observation: most interesting environments (the real world, the internet) are continuous, non-deterministic, dynamic, and only partially observable. Precisely for this reason it is not enough to program an agent as a fixed table of rules.

Example — a thermostat. A not particularly intelligent agent, situated in an environment (a room). Two percepts (“it is cold”, “the temperature is fine”), two actions (switch on, switch off), and a decision unit: cold $\Rightarrow$ switch on, OK $\Rightarrow$ switch off. The environment is nevertheless non-deterministic (someone opens the door).

Example — a software daemon. For instance the Unix `xbiff`: the environment is the operating system, it perceives by calling system services, and its actions are software operations (start a mail client, change an icon). Its decision algorithm is at the level of the thermostat.

2.2 An abstract agent architecture

We now formalise the view of an agent and its interaction with the environment. We assume the environment is discrete — which is not restrictive, as every continuous environment can be discretised to arbitrary precision.

Environment, agent, run

The **environment** can be in one of finitely many discrete states, $E = \{e, e', \dots\}$. The **agent** has a finite set of actions at its disposal, $A = \{a, a', \dots\}$.

A **run** of an agent in an environment is a sequence of alternating environment states and actions:

$$r = e_0 \xrightarrow{a_0} e_1 \xrightarrow{a_1} e_2 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} e_n.$$

Let R denote the set of all finite runs (over E and A), $R^A \subseteq R$ those that end with an action, and $R^E \subseteq R$ those that end with an environment state.

State transformer function of the environment

$$\tau : R^A \rightarrow 2^E$$

The function τ maps a run ending in an action to the set of states the environment may reach after that action. Two consequences follow from this definition:

- **The environment has a history** — the next state depends not only on the last action but on the entire history of the run.
- **The environment is non-deterministic** — the result is a set of states, and we do not know in advance which one will occur.

If $\tau(r) = \emptyset$, the run has ended. In what follows we assume that all runs terminate.

An **environment** is a triple $Env = \langle E, e_0, \tau \rangle$, where e_0 is the initial state.

Agent and system

An **agent** is a function mapping runs that end in an environment state to actions:

$$Ag : R^E \rightarrow A.$$

An agent therefore decides on the basis of the *entire history* of the system, and it decides *deterministically*. Let $\mathcal{AG}$ denote the set of all agents.

A **system** is a pair $\langle Ag, Env \rangle$. The sequence $(e_0, a_0, e_1, a_1, \dots)$ is a *run of agent Ag in environment Env* if and only if

1. e_0 is the initial state of Env ,
2. $a_0 = Ag(e_0)$,
3. for every $u > 0$ we have $e_u \in \tau((e_0, a_0, \dots, a_{u-1}))$ and $a_u = Ag((e_0, a_0, \dots, e_u))$.

The set of all runs of agent Ag in Env is denoted $R(Ag, Env)$.

Functional equivalence of agents

Agents Ag_1 and Ag_2 are **functionally equivalent with respect to Env** if and only if $R(Ag_1, Env) = R(Ag_2, Env)$.

They are **simply functionally equivalent** if they are functionally equivalent with respect to *all* environments.

Functional equivalence matters as a tool for comparing the *expressive power* of architectures: it says that the internal implementation is irrelevant as long as it leads to the same observable runs.

2.3 Purely reactive agents and agents with internal state

Purely reactive (tropistic) agent

$$Ag : E \rightarrow A$$

It reacts only to the current state of the environment and ignores the history.

Relation to the general agent: for every purely reactive agent there exists a functionally

equivalent standard agent; the converse does *not* hold.

Thermostat: $Ag(e) = \text{off}$ for $e = \text{“temperature OK”}$, otherwise $Ag(e) = \text{on}$.

Agent with internal state

Let I be the set of internal states of the agent and Per the set of its percepts. The agent is determined by a triple of functions:

$$\begin{array}{ll} see : E \rightarrow Per & (\text{perception}) \\ next : I \times Per \rightarrow I & (\text{internal state update}) \\ action : I \rightarrow A & (\text{action selection}) \end{array}$$

The cycle: the agent starts in state i_0 ; it perceives the environment ($see(e)$); it updates its state ($i \leftarrow next(i, see(e))$); it selects an action ($a \leftarrow action(i)$); it executes the action and repeats.

Theorem (without proof): For every agent with internal state there exists a functionally equivalent standard agent. Internal state therefore does *not increase expressive power*, but it is essential for the *efficiency* and clarity of the implementation.

The function see at the same time formalises **observability**: if see maps two different states $e_1 \neq e_2$ to the same percept, the agent cannot distinguish them — the environment is only partially observable for it. The extreme cases are $|Per| = |E|$ with see injective (fully observable) versus $|Per| = 1$ (the agent sees nothing).

2.4 How to tell an agent what to do: utility

We do not want agents with hard-wired behaviour. We want to tell the agent *what* to do, not *how*. The standard AI approach is to define the goal *indirectly*, by a measure of success.

Utility

Utility of a state: $u : E \rightarrow \mathbb{R}$. The agent’s goal is to reach states with a high utility value. Evaluating individual states is short-sighted. We therefore introduce the **utility of a run**:

$$u : R \rightarrow \mathbb{R}.$$

This corresponds better to agents that run over the long term. The utility of a run can be derived from states, e.g. as the utility of the *worst* state visited or as the *average* utility of the states visited — which is better depends on the task.

Optimal agent

Let $P(r \mid Ag, Env)$ be the probability of run r of agent Ag in environment Env , where $\sum_{r \in R(Ag, Env)} P(r \mid Ag, Env) = 1$. An optimal agent maximises *expected utility*:

$$Ag_{\text{opt}} = \arg \max_{Ag \in \mathcal{AG}} \sum_{r \in R(Ag, Env)} u(r) P(r \mid Ag, Env).$$

The definition is elegant but **non-constructive**: it does not say how to design such an agent. Moreover, it is often difficult to specify the utility function at all. (A note: money is not a good utility for a human being — the utility of money is non-linear and differs for the rich and the poor.)

Example: Tiledworld. A classical testbed for agent architectures. Agents move around a chess-board in four directions; the board contains holes, obstacles, and tiles, and the goal is to push the tiles into the holes. The environment is *dynamic* — holes, obstacles, and tiles appear and disappear

at random. The utility of a run is

$$u(r) = \frac{N_f}{N_a},$$

where N_f is the number of holes filled by the agent during run r and N_a the number of all holes that appeared during r . The agent has to be able both to *react* to changes (the tile I am pushing has disappeared) and to *exploit opportunities* (a new tile has appeared next to me). Historically, Tileworld was used precisely to investigate the trade-off between reactivity and proactiveness (see agent commitment, section 3.5).

2.5 Predicate task specification

A special case of utility is a mapping into $\{0, 1\}$: a run is either successful ($u(r) = 1$) or not.

Task environment

Let $\Psi : R \rightarrow \{0, 1\}$ be a **predicate specification**: $\Psi(r)$ holds if and only if $u(r) = 1$. A **task environment** is a pair $\langle Env, \Psi \rangle$; it specifies both the properties of the system (through Env) and the criterion for completing the task (through Ψ).

Let $R_\Psi(Ag, Env) = \{r \in R(Ag, Env) \mid \Psi(r)\}$.

When does an agent *solve* a task? There are three readings:

- **Pessimistic**: $R_\Psi(Ag, Env) = R(Ag, Env)$ — *all* runs satisfy Ψ .
- **Optimistic**: $\exists r \in R(Ag, Env) : \Psi(r)$ — *at least one* run.
- **Realistic**: we extend τ with a probability distribution and measure success by the probability $P(\Psi \mid Ag, Env) = \sum_{r \in R_\Psi(Ag, Env)} P(r \mid Ag, Env)$.

Types of tasks

Achievement tasks — “find something”. A set of goal states $G \subseteq E$ is given; $\Psi(r)$ holds if at least one state from G occurs in run r . Typically every classical AI task (search).

Maintenance tasks — “avoid something”. A set of “bad” states $B \subseteq E$ is given; $\Psi(r)$ fails to hold if a state from B occurs in r . Typically games, where B are the losing positions and the environment is the opponent.

Combination: reach a state from G while avoiding the states in B .

2.6 The action selection mechanism — an overview

Action selection mechanism

The procedure by which an agent decides at each moment which of the available actions to perform, on the basis of percepts from the environment and its own internal state. An **agent architecture** is a software architecture that realises such a mechanism — according to Maes (1991) “a particular methodology for building agents that specifies how the agent can be decomposed into a set of components and how these components should interact so that together they answer the question of how the sensor data and the current internal state determine the actions and the future internal state of the agent.”

The following two chapters discuss the main families of action selection mechanisms:

Family	Action selection mechanism	Typical representative
symbolic / deliberative	logical deduction over a database of formulae; planning	deductive agent, STRIPS, BDI, PRS
reactive (symbolic)	a set of “situation $\rightarrow$ action” rules with fixed priorities	subsumption architecture (Brooks)
reactive (connectionist)	activation spreading in a network of modules, the most active one wins	agent network architecture (Maes)
hybrid	layers of different abstraction + arbitration between them	TouringMachines, InteRRaP
coalitional	competition of dynamically arising coalitions of processes for “consciousness”	IDA / global workspace
learned	maximisation of a learned Q -function / stochastic policy	Q-learning, actor-critic (chapter 12)

2.7 Exam summary

- Environment: 4 dichotomies (observability, static/dynamic, determinism, discreteness); real environments are in the hardest variant.
- Formalism: E, A , run $r = e_0 a_0 e_1 \dots e_n$, R^A/R^E , state transformer function $\tau : R^A \rightarrow 2^E$ (hence history + non-determinism), $Env = \langle E, e_0, \tau \rangle$, agent $Ag : R^E \rightarrow A$, system $\langle Ag, Env \rangle$.
- Functional equivalence (with respect to an environment / simple).
- Purely reactive agent $Ag : E \rightarrow A$ (weaker); agent with internal state (*see*, *next*, *action*) — both reducible to a standard agent.
- Utility: $u : E \rightarrow \mathbb{R}$ vs. $u : R \rightarrow \mathbb{R}$; an optimal agent maximises *expected* utility — the definition is non-constructive.
- Task environment $\langle Env, \Psi \rangle$; pessimist/optimist/realist; achievement tasks (G) and maintenance tasks (B).
- Tileworld and the utility N_f/N_a as the standard benchmark of reactivity vs. proactiveness.

3 Deliberative architectures: symbolic reasoning, BDI, PRS

3.1 The classical AI approach and the deductive agent

The classical way in which AI builds an “intelligent system” rests on two pillars:

1. **Symbolic representation** of the environment and of behaviour — here specifically by logical formulae.
2. **Syntactic manipulation** of this representation — that is, logical deduction, theorem proving.

The theory of the agent’s behaviour is then also its program — an *executable specification* that generates concrete actions. This approach runs into two classical problems:

- **The transduction problem:** how to translate the real world into a symbolic representation (computer vision, natural language processing, learning). In Goethe’s words: “All theory, dear friend, is grey, but the golden tree of life springs ever green.”
- **The representation and reasoning problem:** how to represent knowledge and how to reason over it efficiently (theorem provers, planners).

A digression: deduction vs. induction. *Deduction* is the derivation of conclusions that are true whenever the premises are true — a move from the general to the particular (the syllogism: “All men are mortal, Socrates is a man, therefore Socrates is mortal”). *Induction* means that, given true premises, the conclusion is more likely true than false — from the particular to the general. Sherlock Holmes in fact uses induction, even though he calls it deduction; mathematical induction, on the contrary, is deduction.

Deductive agent (the agent as a theorem prover)

Let L be a set of first-order predicate logic formulae and $D = 2^L$ the set of databases of such formulae. The agent's internal state is a database $DB \in D$. Reasoning is realised by inference rules ρ :

$$DB \vdash_{\rho} \varphi \quad \text{— formula } \varphi \text{ is derivable from } DB \text{ using the rules } \rho.$$

The agent is then a triple of functions $see : E \rightarrow Per$, $next : D \times Per \rightarrow D$, $action : D \rightarrow A$, where $action$ is defined by the procedure below.

Action selection as theorem proving.

```
1: function ACTION( $DB : D$ ) :  $A$ 
2:   for all  $a \in A$  do
3:     if  $DB \vdash_{\rho} Do(a)$  then return  $a$ 
4:     end if
5:   end for
6:   for all  $a \in A$  do
7:     if  $DB \not\vdash_{\rho} \neg Do(a)$  then return  $a$ 
8:     end if
9:   end for
10:  return null
11: end function
```

The first loop looks for an action that can be *proved* to be desirable ($Do(a)$ is derived). If there is no such action, the second loop looks for an action that is at least *consistent* with the database (it cannot be proved that it should not be done). If that fails too, the agent does nothing.

Example: the 3×3 vacuum world. Predicates: $In(x, y)$ (the agent is at position (x, y)), $Dirt(x, y)$ (there is dirt at (x, y)), $Facing(d)$ (the agent faces direction d). Rules for deriving an action:

$$\begin{aligned} In(x, y) \wedge Dirt(x, y) &\Rightarrow Do(suck) \\ In(0, 0) \wedge Facing(north) \wedge \neg Dirt(0, 0) &\Rightarrow Do(forward) \\ In(0, 2) \wedge Facing(north) \wedge \neg Dirt(0, 2) &\Rightarrow Do(turn) \quad \dots \end{aligned}$$

The agent thus systematically traverses the world in a “snake” pattern 00–01–02–12–11–10–... Note that cleaning has higher priority than movement (the rule for *suck* is listed first and the other rules have the negation of *Dirt* in their premises).

Pros and cons.

- + Elegant, with clear (and beautiful) semantics; specification = program.
- Practically unusable for non-trivial tasks: proving is slow and need not terminate.
- **The problem of calculative rationality:** the agent derives the optimal action for the state of the environment as it was *at the beginning* of the derivation — meanwhile the environment has changed and the action is no longer optimal. A disaster in a rapidly changing environment.
- The function *see* is hard to design (how to turn an image into formulae, how to represent time).

Nevertheless, individual elements of this approach reappear in later architectures, particularly in BDI.

3.2 Practical reasoning: the BDI architecture

Practical reasoning

Inspired by human decision making. *Theoretical* reasoning (Socrates) leads only to what we think about the world; *practical* reasoning leads to **actions**. It has two phases:

- **Deliberation** — *what we want to achieve* (“I want to finish my master’s degree”).
- **Means-ends reasoning**, that is planning — *how we are going to achieve it* (construct a plan for finishing the degree).

And neither may take too long.

Beliefs — Desires — Intentions

Beliefs — the informational state of the agent, its knowledge about the world. In MAS we deliberately do not call them “knowledge”, in order to stress that they are *subjective*, *need not be true*, and *may change in the future*. They may also contain inference rules that permit forward chaining.

Desires — the motivational state of the agent, the goals or situations it would like to achieve. Desires *may be mutually inconsistent* (I want both to study and to go for a beer).

Intentions — states of the world that the agent has *committed* itself to achieving. Intentions lead to actions: it is for their sake that the agent plans. Intentions **persist** until

1. the agent achieves them, or
2. it comes to believe that they are unachievable, or
3. the reasons that led to them cease to hold.

Moreover, intentions *constrain further deliberation* (I will not reconsider options incompatible with an intention I have adopted) and *influence future beliefs* (I count on my intention succeeding).

The difference between a desire and an intention is illustrated by Bratman (1990): “My desire to play basketball this afternoon is merely a potential influencer of my conduct this afternoon. It must vie with my other relevant desires... In contrast, once I *intend* to play basketball this afternoon, the matter is settled: I normally need not continue to weigh the pros and cons; when the afternoon arrives, I will normally just proceed to execute my intention.”

The deliberation functions

Let $B \subseteq Bel$ denote the agent’s current beliefs, $D \subseteq Des$ its current desires, and $I \subseteq Int$ its current intentions. Deliberation consists of three functions:

- $brf : 2^{Bel} \times Per \rightarrow 2^{Bel}$ — the *belief revision function*, updating beliefs according to a percept;
- $options : 2^{Bel} \times 2^{Int} \rightarrow 2^{Des}$ — generating options (desires) from beliefs and existing intentions;
- $filter : 2^{Bel} \times 2^{Des} \times 2^{Int} \rightarrow 2^{Int}$ — selecting intentions from the options.

Splitting deliberation into *option generation* and *filtering* is essential: generation is cheap and broad, whereas filtering is the actual decision making (it also takes existing commitments into account).

3.3 Planning: STRIPS

Means-ends reasoning / planning

The process of deciding how to achieve a goal (an intention) using the available means (actions).

- **Input:** the goal (= intention I); the current state of the environment (= beliefs B); the actions available to the agent (Ac).
- **Output:** a plan = a sequence of actions after whose execution the goal is satisfied.

STRIPS (Fikes, Nilsson, 1971) models the world by a set of first-order logic formulae. Each action

is described by a *schema* with preconditions and effects (facts added and deleted). The planning algorithm finds the difference between the goal and the current state and reduces it by a suitable action (*means-ends analysis*). This is elegant but not very practical — the algorithm often gets bogged down in low-level details.

Action descriptor, planning problem, plan

An **action descriptor** a is a triple $\langle P_a, D_a, A_a \rangle$: P_a — preconditions, D_a — the delete list, A_a — the add list; for simplicity these are ground atomic formulae (without connectives and variables).

A **planning problem** is a triple $\langle B_0, O, G \rangle$, where B_0 are the initial beliefs, $O = \{\langle P_a, D_a, A_a \rangle : a \in Ac\}$ the descriptors of all actions, and G a set of formulae describing the goal.

A **plan** $\pi = (a_1, \dots, a_n)$, $a_i \in Ac$, determines a sequence of databases

$$B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}, \quad i = 1, \dots, n.$$

A plan is **admissible** if $B_{i-1} \models P_{a_i}$ for every i . A plan is **correct (valid)** if it is admissible and moreover $B_n \models G$.

Planning means, for a given $\langle B_0, O, G \rangle$, finding a correct plan or declaring that none exists.

Example: the blocks world. Predicates: $On(x, y)$ (x is on y), $OnTable(x)$, $Clear(x)$ (nothing is on x), $Holding(x)$ (the arm is holding x), $ArmEmpty$. Actions:

Action	Pre / Del	Add
$Stack(x, y)$	Pre: $\{Clear(y), Holding(x)\}$ Del: $\{Clear(y), Holding(x)\}$	$\{ArmEmpty, On(x, y)\}$
$UnStack(x, y)$	Pre: $\{On(x, y), Clear(x), ArmEmpty\}$ Del: $\{On(x, y), ArmEmpty\}$	$\{Holding(x), Clear(y)\}$
$Pickup(x)$	Pre: $\{OnTable(x), Clear(x), ArmEmpty\}$ Del: $\{OnTable(x), ArmEmpty\}$	$\{Holding(x)\}$
$PutDown(x)$	Pre: $\{Holding(x)\}$ Del: $\{Holding(x)\}$	$\{ArmEmpty, OnTable(x)\}$

Initial state $B_0 = \{Clear(A), On(A, B), OnTable(B), OnTable(C), Clear(C), ArmEmpty\}$, goal $G = \{OnTable(A), OnTable(B), OnTable(C)\}$.

A correct plan: $\pi = (UnStack(A, B), PutDown(A))$. Verification:

- $B_0 \models \{On(A, B), Clear(A), ArmEmpty\}$, so the first action is admissible; after it

$$B_1 = (B_0 \setminus \{On(A, B), ArmEmpty\}) \cup \{Holding(A), Clear(B)\}.$$

- $B_1 \models \{Holding(A)\}$, so the second action is admissible too; after it

$$B_2 = (B_1 \setminus \{Holding(A)\}) \cup \{ArmEmpty, OnTable(A)\}.$$

- $B_2 \models G$, so the plan is correct.

Auxiliary functions over plans

Plan — the set of all plans over Ac ; $pre(\pi)$ — the precondition of a plan; $body(\pi)$ — the body of a plan (the sequence of actions); $empty(\pi)$ — is the plan empty?; $execute(\pi)$ — executes all actions of the plan; $hd(\pi)$ — the first action; $tail(\pi)$ — the rest; $sound(\pi, I, B)$ — plan π is correct with respect to intentions I and beliefs B .

The agent's planning function: $plan : 2^{Bel} \times 2^{Int} \times 2^{Ac} \rightarrow Plan$.

The plan library. The agent need not construct plans online — that is expensive. Very often $plan()$ is implemented as a *library of ready-made plans*: it suffices to traverse it once and check

whether the preconditions of a plan match the current beliefs and whether the effects of the plan match the goal. This idea is the core of the PRS architecture (section 3.6).

3.4 Implementing a BDI agent

The main control loop of a BDI agent:

```

1:  $B \leftarrow B_0; \quad I \leftarrow I_0$ 
2: while true do
3:    $v \leftarrow see()$ 
4:    $B \leftarrow brf(B, v)$ 
5:    $D \leftarrow options(B, I)$ 
6:    $I \leftarrow filter(B, D, I)$ 
7:    $\pi \leftarrow plan(B, I, Ac)$ 
8:   while  $\neg(empty(\pi) \vee succeeded(I, B) \vee impossible(I, B))$  do
9:      $a \leftarrow hd(\pi); \quad execute(a); \quad \pi \leftarrow tail(\pi)$ 
10:     $v \leftarrow see(); \quad B \leftarrow brf(B, v)$ 
11:    if  $reconsider(I, B)$  then
12:       $D \leftarrow options(B, I); \quad I \leftarrow filter(B, D, I)$ 
13:    end if
14:    if  $\neg sound(\pi, I, B)$  then
15:       $\pi \leftarrow plan(B, I, Ac)$  ▷ replanning
16:    end if
17:  end while
18: end while

```

The meaning of the predicates: $succeeded(I, B)$ — the intentions I hold given the beliefs B ; $impossible(I, B)$ — the intentions I cannot hold given B . The inner loop therefore terminates when the plan is exhausted, the goal is achieved, or the goal has become unachievable.

3.5 Agent commitment

Commitment strategies towards intentions

When should an agent drop a goal?

- **Blind commitment** (also *fanatical*) — the agent maintains an intention until it *comes to believe that it has achieved it*. (It never gives up.)
- **Single-minded commitment** — it maintains an intention until it comes to believe that it has achieved it *or* that it is no longer achievable.
- **Open-minded commitment** — an intention persists as long as the agent believes it is still achievable (and still desired) — so it may also be dropped when the motivation changes.

Commitment to a plan versus **commitment to a goal**. The agent in the algorithm above is committed to a single plan; it abandons it when it believes the goal is satisfied, that it is unachievable, or when the plan is empty. But the harder question remains: when should the agent *reconsider the intention itself*?

The classical dilemma: deliberation **takes time** and the environment changes while it is going on.

- An agent that *does not reconsider* its intentions may cling to goals that can no longer be achieved.
- An agent that reconsiders *too often* is so busy reasoning that it gets nothing done.

Meta-level control (Wooldridge, Parsons, 1999): a function $reconsider()$ that is *computationally cheaper* than the reconsideration itself. The quality criterion: $reconsider()$ works well if, whenever it proposes reconsideration, the agent *actually* changes its intention after deliberating (and vice versa).

Bold vs. cautious agent

A **bold agent** — reconsiders its intentions only after completing the entire plan (it never interrupts a plan for the sake of deliberation).

A **cautious agent** — reconsiders after every action of the plan.

The **level of boldness** — how many actions the agent performs between two deliberations.

The **rate of world change** — how many times the environment changes during one cycle of the agent.

Agent effectiveness = the number of intentions achieved / the number of all intentions.

Experimental result (Kinny, Georgeff, in Tileworld):

- If the world changes *slowly*, **bold** agents are more effective — deliberation would be needless overhead.
- If the world changes *rapidly*, **cautious** agents outperform them — plans become obsolete quickly.

This is a concrete quantitative form of the general reactivity vs. proactiveness trade-off.

3.6 The Procedural Reasoning System (PRS)

PRS — *Procedural Reasoning System*

Georgeff et al., 1980s, Stanford. **The first implementation of the BDI architecture** and one of the most successful agent architectures in practice — it has been reimplemented many times: **AgentSpeak/Jason, JAM, JACK, Jadex**.

Practical deployments: OASIS (the air traffic control system in Sydney), SPOC (business process organisation), SWARM (an air combat simulator of the Australian air force).

Components of a PRS agent: a database of *beliefs* (FOL atoms of the Prolog kind), *desires* (goals), a *plan library*, a stack of *intentions*, and an *interpreter* that ties everything together. Sensor data enter the beliefs, and the interpreter issues actions.

A plan in PRS

The agent has a library of ready-made plans representing its *procedural knowledge*. **Nothing is planned from scratch**; plans are only *selected* from the library. A plan has three parts:

- **Goal** (also the *invocation condition* / *cue*) — the condition that is to hold after the plan has been executed; it determines which goal the plan is suited to.
- **Context** — the condition necessary for launching the plan (it must be consistent with the beliefs).
- **Body** — the actions to be performed.

The body of a plan is *not* merely a linear sequence of actions — it may contain further *goals*: “achieve *f*”, “achieve *f* or *g*”, “keep achieving *f* until *g* occurs”. This gives rise to the hierarchical decomposition of goals into subgoals.

The run of the interpreter:

1. Initialisation: the initial beliefs B_0 and a top-level goal.
2. The intention stack contains the current goals in various stages of elaboration.
3. The interpreter takes the intention from the top of the stack and searches the plan library for plans with a matching goal.
4. Among the plans found it selects those whose *context* is consistent with the current beliefs — these constitute the current *options* / *desires*.
5. **Deliberation** = selecting one intention from the options:
 - The original PRS used *meta-plans* — plans about plans, which modified the agent’s intentions. This turned out to be too complicated.

- A more practical variant: each plan carries a numerical *rating* (its expected utility), and the plan with the highest value is selected.
6. The selected plan is executed, which may lead to further intentions being pushed onto the stack.
 7. If a plan fails, the agent selects another intention from the options and continues.

An illustration — Jason/AgentSpeak (an agent that brings its owner beer from the fridge):

```
available(beer,fridge). limit(beer,10).    /* initial beliefs */
too_much(B) :- .count(consumed(_,_,_),B),Q & limit(B,L) & Q > L. /* rule */
+!has(owner,beer)                          /* triggering event */
: available(beer,fridge) & not too_much(beer) /* context */
<- !at(robot,fridge); open(fridge); get(beer); close(fridge); /* body */
    !at(robot,owner); hand_in(beer).
+!at(robot,P) : not at(robot,P) <- move_towards(P); !at(robot,P).
```

The notation `+!g : c <- b` reads: “if the event of adding goal *g* occurs and the context *c* holds, execute the body *b*”. The symbol `!` denotes an *achievement goal* (achieve), `+/-` the addition/removal of a belief or a goal, and `.send` and `.count` are built-in actions.

3.7 Exam summary

- **Deductive agent:** $DB \in 2^L$, $DB \vdash_\rho Do(a)$; two-phase action selection (provable action $\rightarrow$ consistent action $\rightarrow$ nothing). Elegant but impractical; the transduction problem and calculative rationality.
- **BDI:** two phases — deliberation (*brf*, *options*, *filter*) and means-ends reasoning (planning). Beliefs are subjective and fallible, desires may be inconsistent, intentions are commitments that persist and constrain further deliberation.
- **STRIPS:** action descriptor $\langle P_a, D_a, A_a \rangle$, planning problem $\langle B_0, O, G \rangle$, $B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}$; a plan is *admissible* ($B_{i-1} \models P_{a_i}$) vs. *correct* (moreover $B_n \models G$). Be able to work through the blocks world.
- **The BDI loop** and the role of *reconsider* and *sound*.
- **Commitment:** blind / single-minded / open-minded; bold vs. cautious agent; effectiveness = achieved/all intentions; slow environment $\rightarrow$ bold, fast $\rightarrow$ cautious.
- **PRS:** the first BDI implementation; a plan = Goal + Context + Body; a plan library instead of planning; the intention stack; deliberation by meta-plans or by utility; descendants AgentSpeak/Jason, JAM, JACK, Jadex.

4 Reactive and hybrid architectures

4.1 The reactive approach and its premises

During the 1980s and 1990s it became clear that minor adjustments to the symbolic approach were not enough, and alternative AI paradigms arose. Their common denominator is the **rejection of symbolic representation** and of reasoning based on syntactic manipulation. Three key theses:

Three principles of the reactive approach

Situatedness — intelligent behaviour depends on the environment in which the agent is *situated*; it cannot be studied in isolation.

Embodiment — intelligent behaviour is not just logic, it is the product of an agent that *has a body* and interacts physically with the world.

Emergence — intelligent behaviour *emerges* from the interaction of many simple behaviours; it is nowhere explicitly programmed.

Characteristics of reactive agents: they are *behavioural* (the emphasis is on developing and combining individual behaviours), *situated*, and *reactive* (the agent essentially only reacts to the

environment, it does not deliberate). The representation is **subsymbolic**: connectionist networks, finite automata, simple IF–THEN rules.

4.2 Symbolic reactive planning: the subsumption architecture

Subsumption architecture

Rodney Brooks, 1986–1991; the most successful reactive approach. Brooks’s three theses:

1. Intelligent behaviour can be generated *without symbolic representation*.
2. Intelligent behaviour can be generated *without explicit abstract reasoning*.
3. Intelligence is an *emergent property* of certain complex systems.

The intelligence of an agent is realised by a set of simple goal-oriented **behaviours**, for which the following holds:

- Each behaviour is itself an action selection mechanism (a *situation* $\rightarrow$ *action* pair).
- Each behaviour receives percepts and transforms them into actions, is responsible for one partial goal, and has the simple structure of a rule.
- Behaviours run *in parallel* and *compete* with one another for control of the agent.
- Behaviours are arranged into a **subsumption hierarchy**, which defines their priorities (a higher layer “subsumes” = suppresses a lower one).

The action selection mechanism in the subsumption architecture:

```

1: function ACTION(percept  $p$ )
2:    $fired \leftarrow \{ b \in Behaviours \mid p \text{ satisfies the condition of } b \}$             $\triangleright$  behaviours that “fire”
3:   for all  $b \in fired$  do
4:     if there is no  $b' \in fired$  such that  $b' \prec b$  then                                $\triangleright b$  is inhibited by nobody
5:       return the action belonging to behaviour  $b$ 
6:     end if
7:   end for
8:   return null                                                                     $\triangleright$  nothing fired, the agent does nothing
9: end function

```

Formally: a behaviour is a pair $\langle c, a \rangle$, where $c \subseteq Per$ is a condition and $a \in A$ an action; on the set of behaviours R an inhibition relation $\prec \subseteq R \times R$ (a strict total order) is defined. Mind the direction — in Wooldridge and in the lecture course, $b_1 \prec b_2$ means that b_1 **inhibits** b_2 , that is, b_1 is *lower* in the subsumption hierarchy and takes *precedence*. The notation $b_1 \prec b_2 \prec \dots \prec b_n$ thus lists behaviours from the highest priority to the lowest.

Properties: the mechanism is *simple* (although with dozens of behaviours the priorities tend to be tricky) and *very fast* — it can be implemented in hardware and has constant complexity.

Example: Steels’s Mars explorer. The goal is to collect rare rock samples on a distant planet using a swarm of robotic explorers. The means: the base emits a navigation signal (the robots need to do nothing more than detect the *gradient* of the signal); each robot carries radioactive crumbs for *indirect communication* (stigmergy) with the others. Direct communication is not needed.

First iteration — a random walk and the return of a single robot (priority decreases from top to bottom):

```

R1  if I see an obstacle then change direction
R2  if I carry a sample and I am at the base then drop the sample
R3  if I carry a sample and I am not at the base then follow the gradient uphill
R4  if I see a sample then pick it up
R5  if true then move at random

```

Second iteration — better exploration by means of stigmergy. Samples often occur in clusters; a robot that has found one “tells” the others by dropping crumbs on its way home:

R6 **if** I carry a sample **and** I am at the base **then** drop the sample
R7 **if** I carry a sample **and** I am not at the base **then** drop 2 crumbs and follow the gradient uphill
R8 **if** I sense a crumb **then** pick up 1 crumb and follow the gradient downhill

Priorities (in the notation “the left one inhibits the right one”, i.e. from the highest priority): $R1 \prec R6 \prec R7 \prec R4 \prec R8 \prec R5$. A robot following a trail picks up one crumb at a time, so the trail gradually *evaporates* once the cluster has been exhausted. The swarm exhibits cooperative behaviour even though no robot has a model of the world or sends any messages.

4.3 Connectionist reactive planning: the agent network architecture

The subsumption architecture resolves conflicts by *fixed* priority, which is rigid. The connectionist approach replaces fixed priorities with **continuous activation spread through a network** — action selection is the result of the network’s dynamics, not of a fixed ordering. This is “reactive planning”: the agent behaves as if it were planning, but it never constructs an explicit plan.

Agent network architecture / spreading activation networks

Pattie Maes, 1989–1991. The agent is a set of **competence modules**, similar to Brooks’s behaviours. Each module i has:

- **preconditions** c_i — what must hold for the module to be *executable*;
- **effects**: an *add list* a_i and a *delete list* d_i (as in STRIPS);
- an **activation level** α_i and a global **activation threshold** θ .

The modules are connected in a directed graph on the basis of their conditions — a match of pre- and postconditions forms edges, to which further edges representing temporal succession and conflicts are added. At each step activation is propagated; **the executable module with the highest activation is selected**, provided it exceeds the threshold θ .

Three sources and three paths of activation spreading:

- **From the state of the world**: modules whose preconditions are currently satisfied receive activation — “this can be done right now”.
- **From goals**: modules whose add list contains a goal receive activation — “this leads to the goal”.
- **From protected goals**: modules whose delete list contains an already satisfied goal are *inhibited* — “this would destroy what I already have”.

Activation further spreads between modules along three types of link:

- **Successor links**: module x sends activation to module y if y adds something that x needs as a precondition — *forward* chaining (“once I am done, things can go on”).
- **Predecessor links**: a module x that is not executable sends activation to modules that would satisfy its missing precondition — *backward* chaining (“help me so that I can run”).
- **Conflictor links**: module x *inhibits* module y , which destroys its preconditions.

The behaviour of the network is governed by five global parameters: θ (the activation threshold, the only threshold in the system), ϕ (the activation injected into the network by the state of the world), γ (the activation injected by goals), δ (the activation removed by protected goals), and π (the *average* activation level to which the network is normalised after each step, so that the total energy does not change). The ratio γ/ϕ expresses precisely the **proactiveness vs. reactivity** trade-off; θ determines **deliberateness vs. speed** (a high threshold $\rightarrow$ the agent hesitates longer but decides better). If in a given step no executable module exceeds the threshold, θ is lowered by 10 % and the attempt is repeated — the network therefore cannot “freeze”.

How the activation in one step is computed. The update proceeds in four phases: (1) *input from the environment* — each true fact distributes a quantum ϕ evenly among the modules that have it among their preconditions; (2) *input from goals* — each goal distributes γ among the modules that add it, and each protected goal *removes* δ from the modules that would destroy it; (3) *spreading*

between modules — an executable module sends part of its activation forward to its successors (again divided evenly among the recipients), a non-executable module sends activation backward to modules that can satisfy its missing precondition, and conflictor links subtract the corresponding share of activation; (4) *normalisation* — the activations of all modules are rescaled so that their average is π (the total energy of the network remains constant). Then the executable module with activation above the threshold θ is selected; the module that has been executed resets its activation to zero.

A note on the lecture course: the slides present the activation threshold as a property of an individual module (and speak of it as a priority); in Maes’s original model the threshold θ is *global* and the role of the “priority” is played by the current activation level α_i , which changes dynamically.

Comparison with subsumption: the ordering of behaviours is not fixed but arises *dynamically* from the current state of the world and the goals. The network is thus able to generate even multi-step “plan-like” sequences (activation chains backwards from the goal along predecessor links) without any explicit plan data structure ever being created — hence the name *reactive planning*. The price is sensitivity to the parameter settings and the impossibility of guaranteeing optimality or completeness.

4.4 Limitations of reactive architectures

- Reactive agents **have no model of the world**; everything must be derived from the current percepts. Sometimes they therefore have to *change the environment* in order to “remember” a piece of information (the radioactive crumbs) — externalised memory, stigmergy.
- They have only a **short-term view**: they act according to the current state and local information, and it is difficult to take global conditions and long-term goals into account.
- Emergence **is not a good engineering practice**: behaviours have to be tuned experimentally, and both a methodology and any possibility of verification are missing.
- The design is manageable for a few layers; **with dozens of behaviours it becomes intractable** (the number of interactions grows quadratically).

4.5 Hybrid architectures

Neither a purely reactive nor a purely deliberative architecture is ideal. Hybrid architectures combine both:

- A **deliberative / planning component** works at the symbolic level, creating representations and plans.
- A **reactive component** provides immediate responses without complex computation.

The components are usually arranged into **layers** (layered architectures), with the reactive layers as a rule taking *precedence* over the deliberative ones (safety before optimality).

Horizontal vs. vertical layering

Horizontal layering: all layers are connected *in parallel* to both the sensors and the effectors. Each layer is like a separate agent and proposes an action.

- + Conceptually simple: for n kinds of behaviour, n layers suffice.
- The layers influence one another and their proposals conflict; a **mediator function** is needed to resolve the conflict. It is the bottleneck: it has to consider m^n combinations (n layers, m possible proposals) and is a critical point of failure.

Vertical layering: the layers are connected *in series*.

- *One-pass*: a percept enters the lowest layer, passes upwards, and the action emerges from the topmost layer. A natural ordering and hierarchy of behaviours.
- *Two-pass*: percepts travel *bottom-up*, control commands (actions) *top-down*. The flow resembles management in a real company.

- + The need for mediation disappears, and the complexity of the interactions is linear ($n - 1$ interfaces).
- It is not robust: the failure of any one layer brings down the whole agent.

TouringMachines (Ferguson, 1992)

A **horizontal** three-layer architecture for a simulated agent in a traffic environment.

- The **modelling layer** — symbolic models of the agent, the environment, and the other agents; it predicts and resolves conflicts, sets goals, and passes them to the planning layer.
- The **planning layer** — proactive behaviour; it selects from pre-prepared plans (similarly to PRS) and assembles a route plan from them.
- The **reactive layer** — classical reactive *situation* $\rightarrow$ *action* rules, a fast immediate response (obstacle avoidance).

The layers work in parallel and their outputs are unified by a set of **control rules** — a concrete form of the mediator function; these can suppress the inputs and outputs of the layers (*censor* and *suppressor* rules).

InteRRaP (Müller, 1994)

A **vertical two-pass** three-layer architecture.

- The **cooperative planning layer** — social interaction, joint plans with other agents.
- The **local planning layer** — ordinary individual planning.
- The **behaviour-based layer** — reactive behaviour.

Each layer has *its own knowledge base* at the corresponding level of abstraction (knowledge about the world / plans / joint plans and models of the other agents). The layers communicate by two operations: *bottom-up activation* (a lower layer cannot handle the situation and passes it upwards) and *top-down execution* (a higher layer uses a lower one to carry out its decisions).

A real-world example: Stanley (a Volkswagen Touareg R5, Stanford). The autonomous vehicle that won the DARPA Grand Challenge 2005 (212 km across the Mojave Desert); a direct predecessor of today's autonomous vehicles. A layered architecture:

- the sensor layer $\rightarrow$ the abstract perception layer $\rightarrow$ the planning and control layer (the route plan and vehicle control) $\rightarrow$ the vehicle interface layer;
- plus a user interface layer (the panel, starting) and a global services layer (file system, communication, clock).

4.6 Complex architectures: IDA and the global workspace

The last family of architectures goes in the opposite direction to reactive minimalism: it attempts to integrate *everything* into a single agent — action selection, memory, deliberation, emotions.

IDA — *Intelligent Distribution Agent* (S. Franklin)

One of the most complex agent architectures ever built. It arose for a real task of the US Navy — assigning personnel to new billets (hence “distribution”); the architecture itself is distributed as well.

The starting point: global workspace theory (Baars, 1988–97) — one of the theories of consciousness:

- The mind *is* a multiagent system: it consists of many simple, mostly unconscious processes performing specialised operations.
- The processes communicate *rarely* and only through a shared *blackboard*-type memory, organised as an associative array (cf. chapter 8).
- The processes dynamically form higher-order **coalitions**.
- **The action selection mechanism:** the coalition that best matches the current inputs

reaches “consciousness” (the global workspace) and is executed.

- There is a *hierarchy of contexts* representing the world at various levels — the context of goals, of percepts, of the senses, of concepts, and the cultural context.

Assessment: it combines many approaches from MAS and from the rest of AI (action selection, memory, deliberation, emotions), and that is precisely its weakness — many of the decisions are *ad hoc*, and tuning such a heterogeneous collection of models into an optimally cooperating whole is very difficult.

For comparing action selection mechanisms, IDA is useful as a *fourth* type of arbitration: subsumption decides by **fixed priority**, the Maes network by **activation level**, layered architectures by a **mediator function**, and IDA by the **competition of dynamically arising coalitions** for access to the global workspace.

4.7 Comparison of architectures

	Deductive	BDI / PRS	Reactive	Hybrid
Representation	symbolic (FOL)	symbolic (beliefs, plans)	subsymbolic / rules	mixed, per layer
Action selection	proving $Do(a)$	deliberation + plan from a library	priority / activation	layers + arbitration
World model	complete	partial, revisable	none	per layer
Speed	very low	medium	high (const.)	high in reflex
Reactivity	poor	medium (tunable)	excellent	good
Proactiveness	good	excellent	none	good
Design	difficult but formal	systematic	experimental	complicated

4.8 Exam summary

- The reactive approach rests on the triple *situatedness* — *embodiment* — *emergence*; it rejects symbolic representation.
- **Subsumption** (Brooks): a set of simple *situation* $\rightarrow$ *action* behaviours running in parallel, arranged in a fixed priority order. Be able to present Steels’s Mars explorer and the role of stigmergy (the crumbs).
- **Agent network architecture** (Maes) = connectionist reactive planning: modules with precondition, add, and delete lists; activation spreads along successor, predecessor, and conflictor links, and the sources of activation are the state of the world, goals, and inhibition from protected goals; the executable module with the highest activation above the threshold is selected. The parameters γ/ϕ tune proactiveness vs. reactivity.
- Limitations of reactive agents: no world model, short-term view, non-engineering design, an unscalable number of layers.
- **Hybrid**: horizontal (parallel layers + a mediator, m^n combinations, a bottleneck) vs. vertical (serial, one-pass/two-pass, $n - 1$ interfaces, fragile).
- **TouringMachines** = horizontal, 3 layers (modelling, planning, reactive) + control rules. **InteRRaP** = vertical two-pass, 3 layers (cooperative planning, local planning, behaviour-based) + a separate KB per layer. **Stanley** as a real-world deployment.
- **IDA** (Franklin) + *global workspace theory* (Baars): the mind as a MAS of simple unconscious processes communicating through a blackboard; coalitions of processes compete for entry into “consciousness” and the winner is executed; a hierarchy of contexts. The most complex architecture, with many ad hoc decisions.

5 The pathfinding problem

5.1 Formulation and context

Pathfinding is the most common concrete subtask a situated agent has to solve — whether as part of the planning layer of a hybrid architecture, as the action `move_towards(P)` in a BDI plan, or as a service provided to other agents. In a multiagent context two further specifics appear that the single-agent version does not know: the environment is **dynamic** (the map changes, obstacles appear) and the other agents are **moving obstacles** with which it is necessary to coordinate.

The pathfinding problem

We are given a weighted graph $G = (V, E)$ with non-negative edge weights $w : E \rightarrow \mathbb{R}_0^+$, an initial vertex $s \in V$ and a goal vertex $g \in V$ (or a set of goals G_{goal}). We seek a path $s = v_0, v_1, \dots, v_k = g$ minimising $\sum_i w(v_{i-1}, v_i)$.

The graph is usually given *implicitly* by a successor function (a 4-/8-connected grid, a navigation mesh, a visibility graph, a grid of the robot's configuration space).

5.2 A* and related algorithms

Best-first search maintains for every vertex n the values

$g(n)$ = the cost of the best path $s \rightarrow n$ found so far,

$h(n)$ = a heuristic estimate of the cost $n \rightarrow g_{\text{goal}}$,

$f(n) = g(n) + h(n)$ — an estimate of the cost of the best path $s \rightarrow g_{\text{goal}}$ through n .

The A* algorithm (Hart, Nilsson, Raphael, 1968):

- 1: $OPEN \leftarrow \{s\}$ (a priority queue ordered by f), $CLOSED \leftarrow \emptyset$
- 2: $g(s) \leftarrow 0$, $f(s) \leftarrow h(s)$, all other $g \leftarrow \infty$
- 3: **while** $OPEN \neq \emptyset$ **do**
- 4: $n \leftarrow \arg \min_{x \in OPEN} f(x)$; remove n from $OPEN$
- 5: **if** n is a goal **then return** the reconstructed path
- 6: **end if**
- 7: add n to $CLOSED$
- 8: **for all** $m \in \text{succ}(n)$ **do**
- 9: $g_{\text{new}} \leftarrow g(n) + w(n, m)$
- 10: **if** $g_{\text{new}} < g(m)$ **then**
- 11: $g(m) \leftarrow g_{\text{new}}$; $\text{parent}(m) \leftarrow n$; $f(m) \leftarrow g(m) + h(m)$
- 12: insert/update m in $OPEN$ (and remove it from $CLOSED$ if it was there)
- 13: **end if**
- 14: **end for**
- 15: **end while**
- 16: **return** “no path exists”

Admissibility and consistency of a heuristic

A heuristic h is **admissible** if it never overestimates: $h(n) \leq h^*(n)$ for all n , where $h^*(n)$ is the true cost of the cheapest path from n to the goal.

A heuristic is **consistent** (monotone) if for every edge (n, m) the triangle inequality holds:

$$h(n) \leq w(n, m) + h(m), \quad h(g_{\text{goal}}) = 0.$$

Properties: consistency $\Rightarrow$ admissibility. With an admissible heuristic, A* is *optimal* (it finds the cheapest path). With a consistent heuristic, f is moreover non-decreasing along every path and every vertex is expanded *at most once* (there is no need to reopen vertices from $CLOSED$). A* is also *optimally efficient*: no other optimal algorithm using the same heuristic expands

fewer vertices. This claim has to be taken with a grain of salt, however: it holds for vertices with $f(n) < C^*$ (where C^* is the cost of the optimum) — these must be expanded by every optimal algorithm — whereas for vertices with $f(n) = C^*$ it depends on tie-breaking, and a luckier strategy may expand fewer of them.

Special cases and variants:

- $h \equiv 0 \Rightarrow A^*$ degenerates into **Dijkstra's algorithm** (that is, uniform-cost search).
- Typical heuristics on a grid: Manhattan $|\Delta x| + |\Delta y|$ (movement in 4 directions), octile $\max + (\sqrt{2} - 1) \min$ (8 directions), Euclidean (an arbitrary angle).
- **Weighted A***: $f = g + \varepsilon h$, $\varepsilon > 1$ — faster, but ε -suboptimal (the path found is at most ε times more expensive).
- **IDA*** — iterative deepening on f ; memory $O(d)$ instead of exponential.
- **JPS** (*Jump Point Search*) — on uniform grids it skips symmetric paths, giving an order-of-magnitude speed-up without loss of optimality.
- **Theta*** / *any-angle* — allows edges off the grid (line-of-sight checks), giving more realistic trajectories.
- **Hierarchical** planning (HPA*) — the map is divided into clusters and the path is first sought in an abstract graph.

A small example. A 3×3 grid, movement in 4 directions, edge cost 1, start $(0,0)$, goal $(2,2)$, an obstacle at $(1,1)$; the Manhattan heuristic $h(x,y) = |2-x| + |2-y|$ is both admissible and consistent. The start has $f = 0 + 4 = 4$; its successors $(1,0)$ and $(0,1)$ have $g = 1$, $h = 3$, $f = 4$; then $(2,0)$ and $(0,2)$ with $g = 2$, $h = 2$, $f = 4$, then $(2,1)$ and $(1,2)$ with $g = 3$, $h = 1$, $f = 4$, and finally the goal $(2,2)$ with $g = 4$, $h = 0$, $f = 4$. All vertices on the optimal path have $f = 4$ — a typical manifestation of a consistent heuristic that estimates the remainder of the path exactly.

5.3 Pathfinding in a dynamic environment

An agent typically moves and discovers along the way that the map is different from what it thought (a new obstacle, a changed cost). Replanning with A^* from scratch at every step is expensive. The solution is **incremental** search algorithms, which upon a local change recompute only the affected part.

LPA*, D* and D* Lite

LPA* (*Lifelong Planning A**, Koenig, Likhachev, 2001) — an incremental version of A^* for a *fixed* start and goal. In addition to $g(n)$ it maintains for each vertex the value $rhs(n)$ (*right-hand side*), a *lookahead* value derived from the g values of its predecessors:

$$rhs(n) = \begin{cases} 0 & n = s, \\ \min_{p \in pred(n)} (g(p) + w(p, n)) & \text{otherwise.} \end{cases}$$

A vertex is **locally consistent** if $g(n) = rhs(n)$. If the cost of an edge changes, some vertices become inconsistent, are placed into a priority queue, and the algorithm propagates the change only where it is needed.

D* (*Dynamic A**, Stentz, 1994) — the original algorithm for a robot that moves and discovers obstacles. It searches *from the goal towards the start* (so that the values remain usable when the robot moves).

D* Lite (Koenig, Likhachev, 2002) — LPA* searched from the goal, with a correction of the priority queue as the robot moves. Functionally equivalent to D*, but substantially simpler; the *de facto* standard in mobile robotics. Orders of magnitude faster than repeated calls to A^* — typically only a small neighbourhood of the change is recomputed.

Other variants: *Anytime Repairing A** (ARA*) — quickly returns a suboptimal solution and gradually improves it; *AD** — a combination of anytime and dynamic replanning; *Field D** —

interpolation for continuous motion.

And what about continuous space? The preceding algorithms need a graph. In the robot’s *continuous configuration space* (position $\times$ orientation $\times$ joints) the graph is either produced by discretisation or by *sampling*:

- **PRM** (*probabilistic roadmap*) — randomly sample free configurations, connect nearby pairs with a local planner, and search the resulting “road network” in the classical way (A^*). Suitable for repeated queries in a static map.
- **RRT** (*rapidly-exploring random tree*) and **RRT*** — a tree grown incrementally towards random samples; single-shot queries, and it can also handle kinodynamic constraints. RRT is *probabilistically complete* (it finds a solution with probability tending to 1), and RRT* is moreover *asymptotically optimal*.
- **Potential fields** — the goal attracts, obstacles repel; very fast and reactive (close to subsumption), but they get stuck in *local minima*.

What these planners have in common as far as MAS is concerned is that they tend to be the *local* / *low-level* layer of a hybrid architecture, while the coordination between agents (MAPF) is solved over a discrete graph.

An alternative: planning with free space. In an unknown environment the *freespace assumption* is often used: unknown cells are considered traversable, the agent follows the planned path, and as soon as it hits something it replans. This is exactly the scenario for which D^* was designed.

5.4 Multi-agent path finding (MAPF)

Multi-Agent Path Finding, MAPF

Input: a graph $G = (V, E)$ and k agents, where agent i has a start s_i and a goal g_i (all distinct). Time is discrete; at each step every agent either traverses an edge or waits in place.

Conflicts — a solution has to be *conflict-free*:

- a *vertex conflict* — two agents are in the same vertex at the same time;
- an *edge* / “*swap*” *conflict* — two agents exchange their positions in a single step (they pass through each other);
- depending on the variant also a *following* conflict (an agent enters a vertex that its neighbour is just leaving) and *cyclic* rotations.

The objective function: *sum-of-costs* $\sum_i cost_i$, or *makespan* $\max_i cost_i$.

Complexity: finding an *optimal* solution is NP-hard (for both objective functions); deciding whether *some* solution exists is polynomial on general graphs.

Approaches to MAPF split into coupled and decoupled ones:

Type	Principle	Properties
coupled	A^* in the <i>joint</i> state space V^k (a joint state = the positions of all agents)	optimal and complete, but the state space grows exponentially with k ; in practice up to a few agents
decoupled	each agent plans on its own, conflicts are resolved afterwards (priorities, reservations)	fast, scales well, but <i>neither complete nor optimal</i>
hybrid / search over conflicts	searching the space of <i>constraints</i> rather than of positions (CBS, ICTS, M^*)	optimal and yet practically usable for dozens of agents
reductions	translation into SAT, ILP, ASP, or CSP and use of an off-the-shelf solver	optimal, very strong on dense instances

Prioritized planning

The simplest decoupled approach (Erdmann, Lozano-Pérez, 1987):

1. Order the agents by some priority.
2. For $i = 1, \dots, k$: plan the path of agent i with A^* in *space-time* (a state = a pair (vertex, time)) so that it *avoids* the already planned trajectories of agents $1, \dots, i-1$; these are stored in a **reservation table**.

This variant is called **Cooperative A^*** (Silver, 2005); the extension *Hierarchical Cooperative A^** uses an abstract heuristic ignoring the other agents, and *Windowed HCA** plans only over a limited time window (necessary for real-time games).

The weakness: the algorithm is *incomplete* — there exist instances that are solvable but not solvable under any ordering of priorities (classically, two agents in a narrow corridor where one has to step aside into an alcove).

Conflict-Based Search (CBS)

Sharon et al., 2012–2015. A two-level optimal algorithm.

The low level: for each agent an optimal path in space-time is sought using A^* while respecting a set of *constraints* of the form $\langle a_i, v, t \rangle$ (“agent i must not be in vertex v at time t ”), or possibly edge constraints $\langle a_i, (u, v), t \rangle$.

The high level: it searches a *constraint tree* in best-first order by cost:

1. The root: an empty set of constraints, each agent has its individually optimal path.
2. Take the node with the lowest solution cost; check whether the paths are conflict-free. If they are, **return the solution** — it is optimal.
3. Otherwise select the first conflict $\langle a_i, a_j, v, t \rangle$ and *branch* into two children: add the constraint $\langle a_i, v, t \rangle$ to the first one and $\langle a_j, v, t \rangle$ to the second. In each child replan *only* the agent concerned.

The key idea: the joint state space V^k is never searched; branching occurs only on conflicts that have actually arisen, and these tend to be few. CBS is *complete* and *optimal* (sum-of-costs).

Improvements: ICBS/CBSH (stronger heuristics for the high level, classification of conflicts into cardinal/semi-cardinal/non-cardinal), *meta-agent* CBS (agents with many mutual conflicts are merged into a single meta-agent and solved in a coupled way), ECBS (*bounded suboptimal*, orders of magnitude faster at the price of a factor of $1 + \varepsilon$).

An example run of CBS. A corridor $1 - 2 - 3$ with an alcove 4 attached to 2; agent a_1 goes $1 \rightarrow 3$, agent a_2 goes $3 \rightarrow 1$.

- **The root** (an empty set of constraints): the individually optimal paths are $a_1 : (1, 2, 3)$ and $a_2 : (3, 2, 1)$, cost $2 + 2 = 4$. At time $t = 1$ both are in vertex 2 — a vertex conflict $\langle a_1, a_2, 2, 1 \rangle$.
- **The first branching:** the left child adds the constraint $\langle a_1, 2, 1 \rangle$, the right one $\langle a_2, 2, 1 \rangle$; in each of them *only* the agent concerned is replanned. On the left, therefore, $a_1 = (1, 1, 2, 3)$ (it waits), and the cost of the node is $3 + 2 = 5$. But this creates a *swap* conflict: between $t = 1$ and $t = 2$ agent a_1 traverses the edge $1 \rightarrow 2$ and a_2 the same edge $2 \rightarrow 1$. Symmetrically on the right.
- **Further branching** on the edge conflict forces one of the agents to pull into the alcove. The optimal solution has cost **7**: $a_1 = (1, 1, 2, 3)$ for 3 and $a_2 = (3, 2, 4, 2, 1)$ for 4. (Neither cost 5 nor 6 exists: an agent going straight through vertex 2 always causes either a vertex or a swap conflict — try out all four combinations of paths of length 3.)

The example illustrates two typical properties of CBS: the cost of the nodes along a branch is *non-decreasing* ($4 \rightarrow 5 \rightarrow 7$) and *both* agents have to give ground, not just the “yielding” one. Without the alcove the instance is outright **unsolvable** and basic CBS would branch forever — the completeness of CBS is guaranteed only for solvable instances, and unsolvability has to be tested separately.

5.5 Exam summary

- A*: $f = g + h$; *admissibility* $h \leq h^* \Rightarrow$ optimality; *consistency* $h(n) \leq w(n, m) + h(m) \Rightarrow$ every vertex is expanded once. $h \equiv 0$ gives Dijkstra; weighted A* is ε -suboptimal.
- Dynamic environment: LPA* (the values g and rhs , local consistency $g = rhs$), D* and D* Lite (search from the goal, incremental repair), the freespace assumption.
- MAPF: the definition, vertex and edge (swap) conflicts, sum-of-costs vs. makespan; optimal MAPF is NP-hard.
- Decoupled solution: prioritized planning / Cooperative A* with a reservation table, planning in space-time — fast, but incomplete.
- **CBS**: the low level = A* with constraints for a single agent, the high level = binary branching of the constraint tree on the conflict found; optimal and complete, and it avoids the exponential joint space.

6 Knowledge and ontologies

6.1 From an agent to a multiagent system

We now know how to design a single agent. For agents to form a system they have to communicate with one another, and for that they need to resolve:

- **knowledge representation** — in what terms knowledge is expressed;
- **a shared vocabulary** — so that the same word means the same thing (this is the task of *ontologies*);
- **standard messages** — the syntax and semantics of communication (*ACL*);
- **predictable behaviour during communication** — *interaction protocols*;
- **technical problems** — distribution, agent mobility, asynchrony, unreliable communication channels.

6.2 Reasoning about knowledge: epistemic logic and common knowledge

Before turning to the *representation* of knowledge (ontologies), we should say how agents' knowledge is formally *reasoned about*.

Epistemic logic and common knowledge

An agent's knowledge is modelled by a modal operator $K_i\varphi$ ("agent i knows that φ ") with **Kripke possible-worlds semantics**: $K_i\varphi$ holds in world w if and only if φ holds in *all* worlds that agent i cannot distinguish from w (the indistinguishability relation — an equivalence). **Knowledge** is axiomatised by the system **S5** (KT45); the key axiom is **T**: $K_i\varphi \Rightarrow \varphi$ (what I know is true). **Belief** (the operator B_i , cf. BDI and the semantics of FIPA SL) is axiomatised by the system **KD45**, where axiom T is replaced by the weaker **D**: $\neg B_i \perp$ — beliefs must be consistent, but they *may be false*. The difference between knowledge and belief is therefore formally exactly axiom T.

For a group of agents G one defines: $E_G\varphi$ — *everyone* in G knows φ ; **common knowledge** $C_G\varphi \equiv E_G\varphi \wedge E_GE_G\varphi \wedge E_GE_GE_G\varphi \wedge \dots$ (everyone knows that everyone knows that...); *distributed knowledge* $D_G\varphi$ — φ follows from the union of the knowledge of all members of the group.

The coordinated attack (the two generals problem): two generals negotiate a simultaneous attack via an unreliable messenger. No finite number of delivered acknowledgements creates common knowledge of the agreement — after k messages at most $E_G^k\varphi$ holds, never $C_G\varphi$. The lesson for MAS: *unreliable communication never creates common knowledge*; perfect coordination therefore requires either reliable channels or weaker goals (time-bounded or probabilistic variants of common knowledge).

The problem of logical omniscience: in Kripke semantics an agent automatically knows all the logical consequences of its knowledge as well as all tautologies — a real, computationally bounded agent does not. This is the main objection against the straightforward use of epistemic logic as an implementation tool (cf. the difficulties of deductive agents, chapter 3).

6.3 What an ontology is

Ontology

Philosophically (from the Greek *to on* = that which is, *logos* = word): the study of being, according to Aristotle “first philosophy”, a part of metaphysics.

In computer science: an **explicit and formalised description of a part of reality**. Usually a formal declarative description containing a *glossary* (definitions of concepts) and a *thesaurus* (definitions of relations between concepts). An ontology is a kind of vocabulary for storing and exchanging knowledge about a domain in a standard way.

Gruber (1995): “An ontology is a description (like a formal specification of a program) of the concepts and relationships that can formally exist for an agent or a community of agents.” The frequently cited shortened version: *an ontology is an explicit specification of a conceptualisation*.

A motivating example (a dialogue between two people — or two agents). “Have you heard 7777? — No, what is it? — It’s the new CD by the band SRPR, alt.rock with electronics and great lyrics. — I see.” For them to understand each other, they have to share the following: 7777 is an album; it falls into the genres alt.rock and electronica; it has tracks, a performer (SRPR), and authors of the music and the lyrics; SRPR is a band, it has members, and they play instruments. . .

The formalism of an ontology

- **Classes** (concepts) — things that have something in common.
- **Individuals** (instances, objects) — concrete members of classes.
- **Properties** — attributes of classes and individuals.
- **Relations between classes**, in particular the **subclass** (*is-a*) relation — a *transitive* relation.
- Further relations and properties (basic knowledge about the domain).

The **structural part** (classes, relations, axioms) is usually called the *ontology* proper; together with **facts about concrete individuals** it forms a **knowledge base**.

An ontology of ontologies by strength of formalisation (from weak to strong): a vocabulary (a controlled vocabulary of selected terms) → a glossary (definitions of meanings in natural language) → a thesaurus (synonyms) → an informal hierarchy (Amazon, wikis) → a formal *is-a* hierarchy (class subsumption) → classes with properties → value restrictions (“every person has exactly one mother”) → arbitrary logical constraints (which leads to complex inference algorithms).

An ontology of ontologies by scope:

- **Application** ontologies — the most common in practice, hard to reuse.
- **Domain** ontologies — a popular research subject in the semantic web.
- **Upper ontologies** — these are supposed to describe “everything” (Thing, Living thing, ...) and serve as a foundation for domain ontologies: BFO, GFO, UFO; WordNet and IDEAS are not suitable for formal inference. There are methodological objections against the very effort to build a universal ontology (Wittgenstein, *Tractatus logico-philosophicus*).

6.4 Ontology languages

Ontology languages are formal declarative languages for knowledge representation; they contain facts and inference rules and most often rest on first-order logic or on *description logic*.

Frames — a historical predecessor (Minsky), popular in classical AI and expert systems. They correspond to object-oriented programming:

Frame terminology	Object-oriented terminology
Frame	class of objects
Slot	property / attribute of an object
Trigger, accessor, mutator	method

XML was not created for ontologies, but it is used for them: its main advantage is the possibility of defining one's own tags, which naturally represent a vocabulary:

```
<product type="CD">
  <title>7777</title>
  <artist>SRPR</artist>
</product>
```

What it lacks, however, is a defined semantics — it describes only the structure of a document, not its meaning.

RDF — *Resource Description Framework*

The standard tool for knowledge representation for (but not only for) the web. It is **simple**: not very expressive, but with fast inference algorithms.

Everything is expressed by **triples** *subject – predicate – object*, that is, *Predicate(Subject, Object)*. For example, *MarriedTo(Karel, Ja)*, *FatherOf(Karel, Pa)*. A set of triples forms a directed labelled graph (an *RDF graph*); resources are identified by IRIs. Serialisations: RDF/XML, Turtle, N-Triples, JSON-LD. The query language is SPARQL. The extension **RDFS** adds classes, `subClassOf`, `domain`, `range`.

OWL — *Web Ontology Language*

It too originates from the (semantic) web, and exists in versions 1 and 2. It is a *collection of several formalisms* for describing ontologies; the syntax may be XML, RDF, functional notation, Manchester syntax, or Turtle. An attempt at a formal and yet practically usable language.

Its basis is **description logic** — a *decidable fragment* of first-order logic.

The open world assumption (OWA): what we do not know is *undefined* — as opposed to the closed world assumption (databases, Prolog), where everything we do not know is *false*.

Description logic

Three kinds of entity: **concepts** (classes), **roles** (properties, predicates), and **individuals** (objects). An **axiom** is a logical expression about roles and concepts — unlike frames and object-oriented programming, which describe a class *completely*, axioms describe a class only *partially* (and they may come from several sources).

It is a whole family of logical systems according to the constructors allowed. The basic **ALC**: negation, conjunction and disjunction of concepts, and restricted existential and universal quantifiers. Extensions: role hierarchies ($\mathcal{H}$), transitivity ($\mathcal{S}$), inverse roles ($\mathcal{I}$), cardinality restrictions ($\mathcal{N}$, $\mathcal{Q}$), nominals ($\mathcal{O}$).

A knowledge base is divided into the **T-Box** (the terminology — axioms about concepts and roles) and the **A-Box** (the assertional part — facts about individuals).

OWL profiles. *OWL 1 Lite (SHIF)* — the simplest, close to RDF, with many restrictions for the sake of fast machine processing; *OWL 1 DL (SHOIN)* — corresponds to description logic (it makes it possible to express, e.g., the disjointness of classes), is complete and decidable; *OWL 1 Full* — the greatest expressive power, but many problems are undecidable. *OWL 2 (SROIQ)* adds profiles: *EL* (inference in polynomial time, large medical ontologies such as SNOMED), *QL* (queries over a knowledge base, connection to relational databases), and *RL* (axioms in the form of rules).

KIF — Knowledge Interchange Format

Knowledge representation in first-order logic, with a LISP-like prefix syntax. It was designed within the DARPA *Knowledge Sharing Effort* project as the **inner language** of messages (the content), whereas KQML was the *outer* language (the envelope). Examples:

```
(salary 015-46-3946 widgets 72000)
(> (* (width chip1) (length chip1)) (* (width chip2) (length chip2)))
(forall (?F ?T) (=> (and (instance ?F Farmer) (instance ?T Tractor)) (likes
?F ?T)))
(exists (?F ?T) (and (instance ?F Farmer) (instance ?T Tractor) (likes ?F
?T)))
```

DAML+OIL — DARPA Agent Markup Language + Ontology Interchange Language, the direct predecessor of OWL, abandoned in 2006.

6.5 Exam summary

- Epistemic logic: $K_i\varphi$ + possible worlds; knowledge = S5 (axiom T: what I know is true), belief = KD45 (consistency only). Common knowledge $C_G\varphi$ (“everyone knows that everyone knows...”) does not arise from unreliable communication (the coordinated attack); logical omniscience as the main weakness.
- An ontology = an explicit formalised description of a part of reality (glossary + thesaurus); Gruber’s definition. The formalism: classes, individuals, properties, relations (in particular the transitive *is-a*), facts. Ontology + facts = a knowledge base.
- The scale of formalisation from a vocabulary up to arbitrary logical constraints; the division into application, domain, and upper ontologies.
- RDF = *subject–predicate–object* triples, simple and fast; RDFS adds classes.
- OWL = description logic (a decidable fragment of FOL), the *open world assumption*, T-Box vs. A-Box; the Lite/DL/Full profiles (*SHIF*, *SHOIN*), OWL 2 = *SROIQ* + the EL/QL/RL profiles.
- KIF = FOL in LISP-like syntax, the content of a message; KQML = the envelope.

7 Agent communication: speech acts, ACL, protocols

7.1 Why agent communication is different

In object-oriented programming, “communication” means calling a method of an object with parameters — the callee *must* comply. Agents cannot force another agent to do something, nor can they directly change its internal variables. They have to **perform a communicative act** with the aim of

- exchanging information, or
- *influencing* other agents to do something.

The other agents have their own goals and it is entirely up to them what they do with the information, request, or query they receive. This is the fundamental difference: **communication in MAS is an action, not a call.**

7.2 Speech act theory

Speech acts — Austin, 1962

Some parts of the use of language have the character of **actions**, because they change the state of the world in the same way as physical actions do — these are speech acts:

- “I now pronounce you man and wife.”
- “I declare war.”

This is a *pragmatic* theory of language: how speech is used to achieve goals. Typical performative verbs: *request, inform, promise*.

The three layers of a speech act

- The **locutionary act** — *what was said*; the utterance itself. “Make me some tea.”
- The **illocutionary act** — *what was meant*; the locution + the performative meaning (a query, a request, ...). “She asked me for tea.”
- The **perlocutionary act** — *what actually happened*; the effect of the speech act. “She got me to make her some tea.”

For MAS the **illocutionary** part is the crucial one — that is the content of a message (the performative).

Searle elaborated the conditions under which a speech act is “felicitous”. For the act *SPEAKER* requests *HEARER* to perform an action:

- **Normal input/output conditions**: the hearer can hear, it is not a scene in a film. . .
- **Preparatory conditions** (what must hold for the speaker to be able to choose this act): the hearer is able to perform the action; the speaker believes the hearer is able to perform it; it is not obvious that the hearer would perform it even without being asked.
- **Sincerity**: the speaker genuinely wants the action to be performed.

Searle’s categories of speech acts

The older division: *request, advice, statement, promise, ...*

The newer (and standard) division into five classes:

- **Assertives** (representatives) — they inform the hearer (*inform*).
- **Directives** — they request the hearer to perform an action (*request*).
- **Commissives** — the speaker commits to something (*promise*).
- **Expressives** — the speaker expresses a mental state, an emotion (“thank you”).
- **Declarations** — they change the state of affairs (war, marriage).

A speech act always has two components: a **performative verb** (request, query, inform) and a **propositional content** (“the window is closed”).

The planning theory of speech acts (Cohen, Perrault, 1979)

“...modelling [speech acts] in a planning system as operators defined... in terms of the beliefs and goals of speakers and hearers. Speech acts are thus treated in exactly the same way as physical actions.”

The semantics of speech acts is therefore given in the spirit of **STRIPS** (preconditions – delete – add), except that modal operators for *beliefs, abilities, and wants* are used.

Example: Request and Inform.

Request(S,H,A)		Inform(S,H, φ)
<i>cando</i> precondition	$(S \text{ believe } (H \text{ cando } A)) \wedge (S \text{ believe } (H \text{ believe } (H \text{ cando } A)))$	$(S \text{ believe } \varphi)$
<i>want</i> precondition	$(S \text{ believe } (S \text{ want requestInstance}))$	$(S \text{ believe } (S \text{ want informInstance}))$
Effect	$(H \text{ believe } (S \text{ believe } (S \text{ want } A)))$	$(H \text{ believe } (S \text{ believe } \varphi))$

Note that the effect of *Inform* is *not* “the hearer comes to believe φ ”, but only “the hearer comes to believe that the speaker believes φ ”. An autonomous agent is not obliged to believe what anyone tells it — and that is precisely the difference from a method call.

7.3 KQML and FIPA-ACL

KQML — *Knowledge Query and Manipulation Language*

It arose in the 1990s within the DARPA *Knowledge Sharing Effort* (KSE) project together with KIF.

- **KQML** = the *outer* communication language (“the envelope of the letter”); it carries the *illocutionary* part of the message.
- **KIF** = the *inner* language; the propositional content of the message, knowledge representation.

A message is a list consisting of a *performative* and named parameters:

```
(ask-one
  :content (PRICE IBM ?price)
  :receiver stock-server
  :language LPROLOG
  :ontology NYSE-TICKS)
```

Parameters: content, force, reply-with, in-reply-to, sender, receiver, language, ontology.

Performatives: achieve, advertise, ask-about, ask-one, ask-all, ask-if, break, sorry, error, broadcast, forward, recruit-one, recruit-all, reply, subscribe, tell, ...

Criticism of KQML: a *commit* performative was missing, the semantics was not precisely defined, the set of performatives was unsystematic and needlessly large, and the transport mechanism was not standardised.

FIPA-ACL — *Agent Communication Language*

FIPA (the *Foundation for Intelligent Physical Agents*, today an IEEE standards committee) designed ACL as a **simplification and refinement of KQML**: a smaller and more systematic set of performatives and a *formally defined semantics*. A practical implementation: the JADE platform.

The structure of a message (13 fields — the performative and 12 parameters; only performative is mandatory):

Field	Meaning
performative	the type of the communicative act (illocutionary force) — <i>mandatory</i>
sender, receiver, reply-to	agent identifiers (AID)
content	the content of the message itself
language	the language of the content (SL, KIF, ...)
encoding, ontology	the encoding and ontology for interpreting the content
protocol	the interaction protocol the message is part of
conversation-id, reply-with,	matching messages into a conversation
in-reply-to	
reply-by	the deadline by which a reply is expected
(inform	
:sender agent1	
:receiver agent2	
:content (price good2 150)	
:language sl	
:ontology hpl-auction)	

FIPA-ACL performatives (22 in total, the most important ones):

Group	Performatives
passing information	inform, inform-if, inform-ref, confirm, disconfirm
requesting information	query-if, query-ref, subscribe
requesting action	request, request-when, request-whenever, cancel
negotiation	cfp (<i>call for proposals</i>), propose, accept-proposal, reject-proposal
performing actions	agree, refuse, failure
errors, routing	not-understood, proxy, propagate

The semantics of FIPA-ACL: FP and RE

The semantics is defined in the language SL (*Semantic Language*) — a modal logic with the operators $B_i\varphi$ (agent i believes φ), $U_i\varphi$ (it is uncertain), $I_i\varphi$ (it has the intention), Feasible, Done. For every performative the following are given:

- **FP** (the *feasibility precondition*) — what must hold for the sender to be able to perform the act;
- **RE** (the *rational effect*) — the effect for the sake of which it performs the act. The recipient is *not obliged* to bring about the RE (autonomy!) — the RE is only the sender's reason.

For example, for $\langle i, \text{inform}(j, \varphi) \rangle$:

$$\text{FP: } B_i\varphi \wedge \neg B_i(Bif_j\varphi \vee Uif_j\varphi), \quad \text{RE: } B_j\varphi.$$

That is: “I believe φ and I do not believe that j already knows about φ (that it would know whether it holds or not — Bif — nor that it would hold an uncertain opinion about it — Uif); I want j to come to believe φ .” For $\langle i, \text{request}(j, \alpha) \rangle$: FP: $FP(\alpha)[i \setminus j] \wedge B_i \text{Agent}(j, \alpha) \wedge \neg B_i I_j \text{Done}(\alpha)$, RE: $\text{Done}(\alpha)$.

A practical problem: this semantics is unverifiable — it is impossible to check from the outside that an agent really believes what it claims (the *sincerity assumption*). This is a well-known weakness of the mentalistic semantics of ACL; the alternative is a *social* semantics based on public commitments.

7.4 Interaction protocols

A single message is not enough; agents hold **conversations** with a prescribed structure. An interaction protocol is a pattern of admissible sequences of messages; FIPA standardises about a dozen of them (the FIPA-IP specification, AUML notation).

Protocol	Course
FIPA-Request	$I \rightarrow P$: request. $P \rightarrow I$: refuse (end), or agree; after execution inform-done / inform-result, or possibly failure. The reply may also be not-understood.
FIPA-Query	$I \rightarrow P$: query-if / query-ref. P : refuse or agree, then inform with the result or failure.
FIPA-Contract-Net	see below: $\text{cfp} \rightarrow n \times (\text{propose} / \text{refuse}) \rightarrow \text{accept-proposal}$ to the winner + reject-proposal to the others $\rightarrow \text{inform-done} / \text{failure}$.
FIPA-Iterated-Contract-Net	like CNET, but after receiving the bids the initiator may announce <i>another round</i> of cfp (negotiating better terms).
FIPA-Subscribe	I : subscribe; P sends inform every time the monitored value changes, until a cancel arrives.
FIPA-Brokering / Recruiting	I asks a <i>broker</i> , which finds suitable providers, delegates the request to them, and returns the results (or possibly lets them answer directly).
Auction protocols	FIPA-English-Auction, FIPA-Dutch-Auction (see chapter 11).

A note on design. A protocol determines which messages are legal in a given state of the conversation — thereby it introduces *predictability* into otherwise autonomous behaviour and makes it possible to implement an agent as a finite automaton over the states of the conversation (which is exactly what JADE does with classes such as `AchieveREInitiator`, `ContractNetResponder`, and

the like).

7.5 Exam summary

- Agent communication is an *action*, not a method call — the recipient is not obliged to comply.
- Austin: speech acts; locution / illocution / perlocution; Searle: felicity conditions and the five categories (assertives, directives, commissives, expressives, declarations); a performative verb + a propositional content.
- Cohen & Perrault: STRIPS-style semantics of speech acts (pre / effect) with modal operators; be able to write down Request(S,H,A) and Inform(S,H, φ) — the effect of Inform is *H believes that S believes φ* .
- KQML (the envelope) + KIF (the content); parameters and performatives; the criticism (no commit, unclear semantics).
- FIPA-ACL: the structure of a message (performative, sender/receiver, content, language, ontology, protocol, conversation-id, reply-by), 22 performatives, semantics via FP and RE in the language SL; the problem of unverifiability.
- Protocols: FIPA-Request, FIPA-Query, Contract Net (+ the iterated version), Subscribe, Brokering, auction protocols.

8 Distributed problem solving and cooperation

8.1 Coherence and coordination

Agents have different goals, they are autonomous, they operate over time, and they decide at run time; they therefore have to be capable of *dynamic coordination*. In doing so they share two things: **tasks** (task sharing) and **information** (result sharing).

Coherence and coordination

Coherence — how well the system functions *as a whole* (the quality of the solution, the efficiency of resource use, degradation under failures).

Coordination — how well the agents manage to *minimise the overhead* associated with synchronisation: avoiding needlessly duplicated work, conflicts over resources, and unnecessary communication.

A system may be coherent even with poor coordination (the agents do the work three times over, but the result is correct) — at the price of waste.

CDPS — Cooperative Distributed Problem Solving

Lesser et al., 1980s. The cooperation of individual agents in solving a problem that exceeds their individual capabilities (information, resources, computational capacity).

The key assumption: the agents *implicitly share a common goal* and have no conflicts — **benevolence**. The measure of success is the *performance of the system as a whole*; an agent helps the whole even when this is disadvantageous for itself. Benevolence *enormously simplifies* the design of the system.

	Characteristics	Difference
PPS (<i>parallel problem solving</i>)	Bond, Gasser, 1980s; the emphasis is on parallel solving, with homogeneous and simple processors	the agents are not autonomous, this is essentially a parallel algorithm
CDPS	heterogeneous agents with sophisticated capabilities, a shared goal, benevolence	conflicts are not resolved, because they do not arise
MAS	a society of agents with <i>their own</i> goals, which they <i>do not share</i>	it is necessary to address: why and how to cooperate, how to identify and resolve conflicts, how to negotiate and bargain

8.2 Task sharing and result sharing

The general CDPS procedure has three phases:

1. **Problem decomposition** — hierarchical, recursive. The questions are *how* to decompose and *who* performs the decomposition. An extreme variant is the ACTORS model: a new agent is created for every subproblem, down to the level of individual instructions.
2. **Solving the subproblems** — during this the agents typically share information and may need to synchronise their actions.
3. **Solution synthesis** — again hierarchical, the composition of partial results.

Task sharing requires an *agreement* among the agents (who does what) — the typical mechanism is the Contract Net. **Result sharing** may be *proactive* (an agent sends a result it believes will be useful) or *reactive* (it sends it upon request).

The benefits of result sharing

- **Confidence** — independent solutions of the same problem can be compared; agreement increases confidence in the result.
- **Completeness** — the agents share their *local* views and thereby a more global picture of the problem arises.
- **Precision** — sharing may make the overall solution more precise.
- **Timeliness** — the obvious advantage of distribution: a reduction in time.

8.3 The Contract Net

Contract Net Protocol (CNET)

Smith and Davis, 1977–1980. The metaphor of **task sharing by means of contracts** following the model of a market: the agent in need is the *manager*, the others are *contractors*. Four phases:

1. **Recognition** — an agent finds that it has a problem it cannot handle on its own (it lacks the capability, the resources, or the information, or the solution would be worse).
2. **Announcement** — it sends out (typically by broadcast) an announcement of the task: a *task description* (which may even be executable), *constraints* (deadline, price, quality), and *meta-information* (who may apply).
3. **Bidding** — the recipients decide whether they have the interest and the capabilities, and send a bid (a *tender*) with a price / time / quality.
4. **Awarding and expediting** — the manager selects the winner and concludes a contract with it; the contractor performs the task and returns the result.

Properties: simple, decentralised, robust against failures; a contractor may itself become the manager of a subtask, which leads to *hierarchical cascades of subcontracts*. It distributes the load dynamically. It is the most frequently implemented coordination mechanism of all (in FIPA it is standardised as FIPA-Contract-Net; in JADE it is available as a ready-made behaviour).

Limits: broadcasting is expensive with many agents; it assumes benevolence (a contractor does not lie about its capabilities) — otherwise it is necessary to use auction mechanisms that are robust against manipulation (chapter 11).

8.4 Blackboard systems

Blackboard system (BBS)

The very first scheme for cooperative problem solving (HEARSAY-II, speech recognition, 1970s). Results are shared through a common data structure — the **blackboard**.

The metaphor: several experts sit around a blackboard and can both write on it and read from it. Tasks appear on the blackboard dynamically; as soon as one of the experts sees a task it knows how to solve, it writes a partial solution on the blackboard. This continues until the

solution of the whole task appears on the blackboard.

The roles:

- The **blackboard** — shared memory; typically structured into several *levels of abstraction* (a hierarchy of goals and actions). The formalism for representing information is essential.
- The **experts** (*knowledge sources*) — agents solving a particular type of task; they react to the goals on the blackboard, take their goal from the blackboard, and once selected they perform an action. They have a list of capabilities and an efficiency.
- The **arbiter** (the *control component*) — it selects which expert may approach the blackboard. It may be purely reactive, or it may reason about plans and maximise expected utility. It is responsible for solving the problem at a higher level.

Mutual exclusion over the blackboard is necessary — the bottleneck of the system.

Pros and cons of BBS:

- + A simple mechanism for coordination, cooperation, and the sharing of both tasks and results.
- + The experts *need not know about each other* and yet they cooperate (*loose coupling*); adding an expert requires no change to the others.
- + Messages on the blackboard can be rewritten — delegating tasks, creating subtasks, changing experts.
- All agents must have access to the blackboard and share the same architecture; it tends to be “crowded” around the blackboard (distributed hash tables partly address this).

Example: BBWar. The blackboard is a hash table mapping required capabilities to tasks; “open missions” are published on the blackboard. The experts form a hierarchy: a *commander* agent looks for tasks of the type “conquer a city” (ATTACK-CITY) and converts them into several tasks “attack a position” (ATTACK-LOCATION); soldiers of various types then pick from the blackboard the missions they have the capabilities for.

Example: FELINE (Wooldridge, Jennings, 1990) — a distributed expert system from the time before KQML and ACL. Knowledge sharing and the distribution of subtasks; each agent is a rule-based system that maintains: *Skills* — “I can prove/refute the following...” and *Interests* — “I am interested in whether the following holds...”. Communication is a triple (sender, recipient, content), where the content is a hypothesis + a speech act (request, response, inform).

8.5 Distributed constraints (DisCSP and DCOP)

Both CNET and the blackboard are *protocols*. Alongside them stands the formal branch of distributed problem solving, in which the task is given as a system of constraints spread among the agents.

DisCSP and DCOP

A **distributed CSP** (distributed constraint satisfaction problem): the variables $x_1, \dots, x_n$ with domains D_i are divided among the agents (typically one variable per agent) and the constraints link the variables of *different* agents. An assignment satisfying all the constraints is sought; no agent sees the whole problem, and each communicates only with its neighbours in the constraint graph.

DCOP (distributed constraint optimization) is the optimisation variant: the constraints are *soft*, i.e. given by cost functions $f_{ij}(x_i, x_j)$, and $\sum_{ij} f_{ij}$ is maximised — that is, social welfare (chapter 9).

Algorithms: *ABT* (*asynchronous backtracking*, Yokoo) — the agents are ordered and send each other *ok?* messages (their assignment downwards) and *nogood* messages (back upwards upon a conflict), where the learned *nogoods* prevent cycling; *ADOPT* (asynchronous, with bounds, optimal), *DPOP* (dynamic programming over a pseudo-tree — linearly many messages, but

exponentially large ones), and *Max-Sum* (message passing over a factor graph, approximate but very scalable).

Typical applications: meeting scheduling, assigning tracking tasks to sensors, frequency allocation, coordination in MAPF. Unlike in CDPS, benevolence is *not* required here: DCOP can be combined with the mechanisms of chapter 11.

8.6 Joint intentions and social laws

What exactly does it mean for a team of agents to do something *jointly*? The answer is the multiagent generalisation of commitment from chapter 3.5:

The theory of joint intentions (Cohen–Levesque)

A team G has a **joint persistent goal** φ if: (1) all members believe that φ does not hold yet and want it to hold; (2) this is mutually known among them; (3) each member holds the goal until it learns that φ is *achieved*, *unachievable*, or *irrelevant* — and whoever finds this out takes on the **commitment to let the others know** (the state of the team becomes that agent’s individual goal to announce). It is precisely the communicative obligation in point (3) that distinguishes team activity from a chance coincidence of individual intentions (Searle’s example: runners who all dash for the same bandstand when it starts raining are not a team). Practical models of team cooperation build on this theory: Jennings’s model of *joint commitments* with *conventions* — explicit rules stating when an agent may abandon a commitment and whom it must inform — and the **STEAM** system (Tambe); an alternative formalisation is *SharedPlans* (Grosz–Kraus).

Social laws and conventions (Shoham–Tennenholtz): coordination need not take place at run time through communication — it can be ensured *offline* by rules that constrain the actions of all agents in advance (the model example: “drive on the right”). A social law is *useful* if it leaves every agent the possibility of achieving its goals; the synthesis of a useful social law is NP-hard in general. Conventions may also *emerge* from below through repeated interaction (cf. Axelrod’s tournaments, chapter 9). Wooldridge summarises four basic coordination techniques: **organisational structuring** (roles and hierarchies, cf. Gaia, chapter 13), **contracting** (CNET), **multiagent planning**, and **negotiation** (chapter 11).

8.7 Agent interaction in the environment

Agents can help and hinder one another even *without communication*, simply by affecting the environment:

- Robots can carry a brick only if they push from two sides simultaneously (a positive interaction, requiring synchronisation).
- Robots crowd at a door and cannot open it (a negative interaction, requiring coordination).

Agents can form communities, hierarchies, and hostilities. **It is necessary to know the types of interaction** in a specific MAS — otherwise effective control mechanisms cannot be designed. The simplest model case, which the following chapter deals with, is the interaction of two rational (selfish) agents in an environment resembling a game.

8.8 Exam summary

- Coherence (the performance of the whole) vs. coordination (minimising synchronisation overhead).
- CDPS: benevolence + a shared goal; distinguish it from PPS (homogeneous processors, a parallel algorithm) and from MAS (own goals, conflicts, negotiation).
- The three phases of CDPS: decomposition — solving the subproblems — synthesis. Task sharing (agreement) vs. result sharing (proactive/reactive) and its four benefits: confidence, completeness,

precision, timeliness.

- CNET: recognition — announcement — bidding — awarding/expediting; the manager and the contractor, hierarchical subcontracts; standardised as FIPA-Contract-Net.
- BBS: the blackboard (a hierarchy of abstraction, mutex), the experts (skills), the arbiter (selecting an expert); loose coupling between the experts; access to the blackboard as a bottleneck.
- DisCSP / DCOP: variables and constraints divided among the agents, nobody sees the whole problem; ABT (ok? / nogood), ADOPT, DPOP, Max-Sum; DCOP maximises social welfare.
- A joint persistent goal: the team holds the goal until it is achieved / unachievable / irrelevant, and whoever finds this out must announce it to the others; social laws = offline coordination by constraining actions (“drive on the right”).

9 Agent interaction and game theory

9.1 The selfish agent

Why should an agent be honest about its capabilities? Why should it complete an assigned task? If the system is homogeneous (designed entirely by us), benevolence is a good strategy. Usually, however, the system contains agents with different interests — a conflict between the common goal and the goals of individuals (e.g. air traffic control: everybody wants safety, but every airline wants to land first). Sometimes it is not even clear what the common interest is. In that case it is better to consider **self-interested** agents.

Self-interested agent

The set of possible outcomes $O = \{o_1, o_2, \dots\}$ is *common* to all agents, but the preferences are not: each agent i has its own **utility function** $u_i : O \rightarrow \mathbb{R}$, which orders the outcomes.

Strategic thinking: maximise your utility under the assumption that everybody else acts rationally as well. This does *not* maximise the common utility, but it is a *robust* strategy.

A note: money is not a good utility for a human being — the utility function of money is non-linear and differs for the rich and the poor.

9.2 Normal-form games

Two-player game in normal form

Agents i and j with action sets A_i, A_j . The outcome is determined by both: $e : A_i \times A_j \rightarrow O$. An agent chooses a **strategy** s_i :

- **pure** — a deterministic choice of a single action;
- **mixed** — a probability distribution over pure strategies (a random choice).

A game is written down as a **payoff matrix**: rows = the strategies of player i , columns = the strategies of player j , and a cell contains the pair u_i/u_j .

In general: a game in normal form is a triple $\langle N, (A_i)_{i \in N}, (u_i)_{i \in N} \rangle$, where N is the set of players.

Dominance

A strategy s_i **strictly dominates** a strategy s'_i if it gives agent i a *strictly* better outcome against *all* strategies of the opponent; it **weakly dominates** it if it gives an outcome at least as good against all of them and a strictly better one against at least one (this is also how the lecture course defines dominance). A strategy is **dominant** if it dominates all the other strategies of agent i .

The difference is not academic: truthfulness in the Vickrey auction is only *weakly* dominant (chapter 11).

A rational agent plays a dominant strategy if one exists. *Iterated elimination of strictly dominated strategies* (IESDS) is the basic solution technique: nobody will ever play a dominated strategy,

so it can be struck out of the matrix, which may render further strategies dominated. When eliminating *strictly* dominated strategies the order does not matter and the result is unique; with *weakly* dominated strategies the order **does** matter and some equilibria may be lost.

Nash equilibrium

A pair of strategies (s_1^*, s_2^*) is a **Nash equilibrium** if:

- when agent i plays strategy s_1^* , it is best for agent j to play s_2^* , and at the same time
- when agent j plays strategy s_2^* , it is best for agent i to play s_1^* .

That is, s_1^* and s_2^* are *mutual best responses*. In general, for n players: a profile $s^* = (s_1^*, \dots, s_n^*)$ is a NE if for every i and every strategy s_i of that player

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*),$$

where s_{-i} denotes the strategies of all the others. **No player has a unilateral reason to deviate.**

A naive search for equilibria for n agents and m strategies requires going through m^n profiles. This definition concerns *pure* strategies — not every game has a NE in pure strategies (rock–paper–scissors) and some games have several.

Theorem 9.1 (Nash’s theorem, 1950). *Every game with a finite number of players and a finite number of strategies has at least one Nash equilibrium in mixed strategies.*

Computational complexity. Finding a NE is a *total* search problem (a solution always exists, the only question is how to find it). Since 2006 we have known that it is **PPAD-complete** — very roughly speaking “somewhat less hopeless than NP-completeness” (NP-completeness does not even make sense for a problem that always has a solution). For three and more players this was proved by Daskalakis, Goldberg, and Papadimitriou; shortly afterwards Chen and Deng supplied the remaining and hardest case of *two* players.

Pareto efficiency and social welfare

A strategy profile is **Pareto optimal** (Pareto efficient) if there is no other profile that would improve the outcome of one agent without worsening the outcome of some other one. A non-Pareto-optimal solution can therefore be improved “for free”.

The **social welfare** of a profile is the sum of the utilities of all agents: $sw(o) = \sum_{i \in N} u_i(o)$. Maximising social welfare makes sense in cooperative scenarios — when the agents are on the same team, have a single owner, or solve a single task; the more homogeneous the system, the better. The maximum of social welfare is always Pareto optimal, but not conversely.

9.3 Classical games

The Prisoner’s dilemma

$i \backslash j$	C (cooperate)	D (defect)
C	3/3	0/4
D	4/0	1/1

C = *cooperate*, cooperate with one’s accomplice; D = *defect*, inform on him and get a lower sentence. The canonical notation for the payoffs: R (*reward* for mutual cooperation), P (*punishment* for mutual defection), T (*temptation* — I defect against a cooperator), S (*sucker’s payoff* — I am betrayed). The conditions of the prisoner’s dilemma:

$$T > R > P > S \quad \text{and often in addition} \quad 2R > T + S.$$

- $R > P$: mutual cooperation is better than mutual defection.
- $T > R$ and $P > S$: *defection is a dominant strategy for both agents.*

- (D, D) is the **only Nash equilibrium**.
- (D, D) is at the same time the **only non-Pareto-optimal** outcome of the game.
- (C, C) maximises social welfare ($3 + 3 = 6$).

The rationality of each individual therefore leads to the collectively worst outcome.

Consequences of the prisoner's dilemma:

- **The tragedy of the commons:** a shared resource (pasture, fisheries, the atmosphere) is depleted because every individual maximises their own benefit.
- The question of *what being rational actually means* — and whether people are rational (experimentally, people cooperate more often than the theory predicts).
- **The shadow of the future:** in the *iterated* prisoner's dilemma the situation changes.

The iterated prisoner's dilemma and Axelrod's tournaments

If the PD is played *a finite number of times and both players know it*, backward induction yields D in every round (in the last round there is no reason to cooperate, hence none in the next-to-last either, ...). If it is played *infinitely* (or with an unknown end, i.e. with a probability of continuing), cooperation becomes rational — the *folk theorem*: every payoff profile better than the minimax can be supported as an equilibrium of the repeated game.

In 1980 Robert Axelrod organised tournaments of strategies for the iterated PD. The winner (twice) was the simplest strategy submitted, **Tit For Tat** (TFT): *cooperate in the first round, then always repeat the opponent's last move*. From the results Axelrod derived four properties of successful strategies:

- being **nice** — do not defect first;
- being **retaliating** — respond to defection immediately;
- being **forgiving** — forgive an opponent who returns to cooperation;
- being **non-envious** — do not strive to beat a *particular* opponent; TFT never scored more points than its counterpart, and yet it won the tournament.

As a fifth property Axelrod lists **clarity**: a strategy must be legible to the opponent, otherwise no cooperation can be established with it. The weakness of TFT: under noise (a move executed incorrectly) two TFT players get stuck in mutual retaliation — hence the variants *Generous TFT* (which occasionally forgives) and *Tit for Two Tats*.

Coordination game

$i \backslash j$	Ballet	Wrestling
Ballet	1/2	0/0
Wrestling	0/0	2/1

Also known as the *Battle of the Sexes* (BoS) or *Bach or Stravinsky*. The payoffs favour *agreement* of strategies, but the players differ in which agreement they care about more.

- Nash equilibria in pure strategies: (Ballet, Ballet) and (Wrestling, Wrestling).
- Social welfare and Pareto optimality: again both of these pairs.
- **The mixed NE:** each player plays their *more preferred* option with probability $2/3$ and the less preferred one with $1/3$. The expected utility in the mixed equilibrium is only $2/3$ — *less* than in either pure equilibrium.

Computing the mixed NE (the general recipe for 2×2 games): player j chooses the probability q of playing Ballet so that player i is *indifferent* between their actions:

$$u_i(\text{Ballet}) = 1 \cdot q + 0 \cdot (1 - q), \quad u_i(\text{Wrestling}) = 0 \cdot q + 2 \cdot (1 - q).$$

The equality $q = 2 - 2q$ gives $q = 2/3$, i.e. player j plays Ballet (their preferred option) with probability $2/3$. Symmetrically, player i plays Wrestling with probability $2/3$. The expected utility of player i is then $1 \cdot \frac{2}{3} = \frac{2}{3}$. **The key rule: in a mixed NE each player chooses their probabilities so as to make the opponent indifferent.**

Anti-coordination game

$i \backslash j$	Chicken (swerve)	Dare (hold course)
Chicken	5/5	1/6
Dare	6/1	0/0

Also known as *Chicken*, *Hawk–Dove*, or *Snowdrift*. The payoffs favour the choice of *different* strategies. In coordination games sharing a resource benefits everyone; in anti-coordination games sharing is costly (a collision of cars).

- Nash equilibria in pure strategies: (Chicken, Dare) and (Dare, Chicken).
- The maximum of social welfare: (Chicken, Chicken) — $5 + 5 = 10$, which is *not* a NE (both players benefit from deviating to Dare: $6 > 5$).
- **The symmetric mixed NE:** if the opponent plays Chicken with probability p , then $u_i(\text{Chicken}) = 5p + 1(1 - p) = 1 + 4p$ and $u_i(\text{Dare}) = 6p$; from the indifference $1 + 4p = 6p$ it follows that $p = 1/2$. Both therefore play both actions with probability $1/2$ and the expected utility is 3 — less than the 5 under (Chicken, Chicken), but nobody can improve unilaterally.

An addendum: a game with no pure NE. *Matching pennies* (and likewise rock–paper–scissors) is a zero-sum game:

$i \backslash j$	Heads	Tails
Heads	1/ − 1	−1/1
Tails	−1/1	1/ − 1

No pure NE exists. The mixed NE: both play $(1/2, 1/2)$, with expected utility 0. The motif is analogous to the Condorcet paradox from the following chapter: individually rational choices have no “stable” deterministic outcome.

9.4 Cooperative (coalitional) games

Everything so far has concerned **non-cooperative** games: the basic unit is the *individual* and their strategies, and agreements are not enforceable. **Cooperative** game theory changes the unit to the *group* and asks: who joins forces with whom, and how do they divide the joint gain? For MAS this is a formal model of team formation and reward division.

Coalitional game with transferable utility

A pair $\langle N, v \rangle$, where N is a set of agents and the **characteristic function** $v : 2^N \rightarrow \mathbb{R}$ assigns to every *coalition* $C \subseteq N$ the value its members can secure by their own efforts ($v(\emptyset) = 0$). The utility is *transferable*: the value $v(C)$ can be divided arbitrarily among the members (“in money”).

A game is **superadditive** if $v(C \cup D) \geq v(C) + v(D)$ for disjoint C, D — in that case it always pays off to form the *grand coalition* N , and only the question of the division remains.

A **solution** is a vector $(x_1, \dots, x_n)$ with $\sum_i x_i = v(N)$ (*feasibility*).

The core and the Shapley value

The core — the set of divisions that *no* coalition wants to leave:

$$\text{core} = \left\{ x : \sum_{i \in N} x_i = v(N) \wedge \forall C \subseteq N : \sum_{i \in C} x_i \geq v(C) \right\}.$$

It is the analogue of the Nash equilibrium for groups (*stability*). The core may, however, be **empty** — in which case no stable division exists — and it may also be very large, so it does not tell us which division is *fair*.

The Shapley value (1953) — the answer to the question of fairness. An agent receives its *average marginal contribution* over all the orders in which the coalition can be assembled:

$$\varphi_i(v) = \sum_{C \subseteq N \setminus \{i\}} \frac{|C|! (|N| - |C| - 1)!}{|N|!} (v(C \cup \{i\}) - v(C)).$$

It is the *only* division satisfying simultaneously: efficiency ($\sum_i \varphi_i = v(N)$), symmetry (interchangeable agents receive the same), the *dummy* property (an agent with a zero marginal contribution receives nothing), and additivity.

Example (gloves): agents 1 and 2 each have a left glove, agent 3 a right one; a pair has value 1, a single glove 0. Thus $v(\{1, 3\}) = v(\{2, 3\}) = v(N) = 1$, and 0 otherwise. The core contains a *single* point $(0, 0, 1)$ — agent 3 takes everything, because agents 1 and 2 are dependent on it. The Shapley value is $(1/6, 1/6, 2/3)$ — fairer, but unstable.

The computational side (why this interests MAS): the characteristic function has 2^n values, so even its *representation* is a problem (compact languages and games over graphs are used); deciding the non-emptiness of the core and computing the Shapley value are both hard in general, and the *coalition structure generation* problem (finding the optimal partition into coalitions when the game is not superadditive) is NP-hard.

9.5 Exam summary

- A selfish agent maximises *its own* expected utility under the assumption that the others do the same; this does not maximise the common utility.
- Normal-form games, pure vs. mixed strategies, dominance and dominant strategies, iterated elimination of dominated strategies.
- **Nash equilibrium** = mutual best responses, nobody has a reason to deviate unilaterally; a naive search takes m^n ; **Nash's theorem** (existence in mixed strategies); finding one is PPAD-complete.
- **Pareto optimality** (one agent cannot be made better off without making another worse off) vs. **social welfare** (the sum of utilities); the maximum of sw is Pareto optimal, but not conversely.
- **The prisoner's dilemma**: $T > R > P > S$; D dominant; (D, D) the only NE and the only non-Pareto-optimal outcome; (C, C) maximises sw. The tragedy of the commons. The iterated PD, the shadow of the future, Axelrod and TFT (nice, retaliating, forgiving, clear).
- **Coordination game** (BoS): two pure NE + a mixed one, $2/3$ – $1/3$, with worse utility; **anti-coordination** (Chicken): two asymmetric NE, and sw is maximised by a non-equilibrium profile.
- Be able to compute the mixed NE for 2×2 : choose the probabilities so as to make the *opponent* indifferent.
- **Cooperative games**: the characteristic function $v : 2^N \rightarrow \mathbb{R}$, superadditivity and the grand coalition; **the core** (no coalition wants to leave, may be empty) vs. **the Shapley value** (the average marginal contribution; efficiency, symmetry, dummy, additivity). The gloves example: core $(0, 0, 1)$, Shapley $(1/6, 1/6, 2/3)$.

10 Social choice and voting

10.1 Preference aggregation

The previous chapter dealt with how agents behave when they have conflicting interests. Now we ask the converse question: how to *derive* a collective decision from individual preferences. This is the subject of **social choice theory**.

Social welfare vs. social choice

A **social welfare function** aggregates individual preferences into a *complete ordering* of the candidates $>_{sw}$.

A **social choice function** selects only *one* winner (or a set of winners).

Example: voters $A : o_2 > o_1 > o_3$, $B : o_3 > o_2 > o_1$, $C : o_2 > o_3 > o_1$. Social welfare: $o_2 > o_3 > o_1$. Social choice: o_2 .

Ballot and preference schedule

A **ballot** — how a voter expresses their preferences. Traditionally: “pick your favourite from the list”. A **preference ballot** — the voter ranks *all* the options by preference.

The individual ballots are combined into a **preference schedule**; from it a **voting scheme** determines the winner.

10.2 Voting systems

- **Majority** — the winner gets more than 50 % of the votes; trivial for two candidates.
- **Plurality** — a candidate gets a point for every first place, and the one with the most points wins; in practice the ballot contains only the first choice, and in the event of a tie a *run-off* between the two best follows.
- **Plurality with elimination** (*instant runoff*, IRV) — repeatedly eliminate the candidate with the fewest first places and *redistribute* their votes to the next choice of the voters concerned, until the winner has a majority. It makes use of the already recorded preferences without a second round and suppresses insincere voting (the Australian House of Representatives, the IOC).
- **Sequential majority** — the candidates play tournament rounds in pairs and the winner advances (a knockout bracket). *The order of the matches affects the result* (agenda setting).
- **The Condorcet method** — simulate *all* pairwise contests; expensive, and it need not produce a winner.
- **The Borda count** — every voter submits complete preferences and a candidate gets $N - 1$ points for a first place, $N - 2$ for a second, $\dots$, 0 for the last, and the points are summed. This is *consensual* rather than majoritarian voting (Slovenia, Nauru, Iceland, academic institutions).
- **Slater’s system** — from the pairwise majority contests build a tournament graph and look for an *acyclic* ordering minimising the number of reversed edges. **NP-hard**.
- **Combinations with Borda** (all of which elect the Condorcet winner if one exists): *Black* (the Condorcet method, otherwise the Borda winner); *Baldwin* (repeatedly eliminate the candidate with the lowest Borda score and recompute); *Nanson* (repeatedly eliminate all candidates below the average).

The plurality winner need not be a majority winner. Example: o_1 — 40 %, o_2 — 30 %, o_3 — 30 %. Candidate o_1 wins, but 60 % of the voters did not want them. This leads to **tactical (insincere) voting**: if o_2 and o_3 are similar (from the same party), the voters of o_2 will vote for o_3 in order to defeat o_1 . In practice plurality is used with the requirement of an absolute majority and a possible second round between the two best (the Czech presidential and Senate elections).

The Condorcet paradox

There are situations in which *whichever outcome we choose, a majority of voters will be dissatisfied with it*.

Example: $V_1 : A > B > C$, $V_2 : B > C > A$, $V_3 : C > A > B$. Pairwise contests: A beats B (2:1), B beats C (2:1), C beats A (2:1). **The collective preference is cyclic**, even though all the individual preferences are linear. There is no *Condorcet winner*.

The situation is analogous to the non-existence of a Nash equilibrium in pure strategies in rock–paper–scissors: individually rational (linear) preferences compose into a cycle that has no stable deterministic top. This is not a formal reduction, but it is the same motif.

The Marquis de Condorcet (1743–94), a French philosopher and mathematician, active in the French Revolution, died in prison. He proposed several *criteria of fairness* and thereby founded the mathematical theory of voting.

10.3 A worked example: one preference schedule, four winners

A typical exam problem: given a preference schedule, compute the winner under the individual systems. The following schedule is chosen so that *four* systems give *four different* winners. Candidates A, B, C, D , and 17 voters in total:

Number of voters	5	6	4	2
1st place	A	C	D	B
2nd place	D	B	B	A
3rd place	B	D	A	C
4th place	C	A	C	D

1. Plurality. First places: $C = 6$, $A = 5$, $D = 4$, $B = 2$. **Winner** C (nobody has an absolute majority, i.e. 9 votes).

2. Plurality with elimination (IRV).

- Round 1: nobody has 9 votes; we eliminate B (2 first places). Its voters ($B > A > C > D$) have A as their next choice: $A = 5 + 2 = 7$, $C = 6$, $D = 4$.
- Round 2: still nobody has 9; we eliminate D (4). Its voters ($D > B > A > C$) have, after the elimination of B , the next choice A : $A = 7 + 4 = 11$, $C = 6$.
- A has 11 out of 17, i.e. a majority — **winner** A .

3. The Borda count (4 candidates, hence 3/2/1/0 points):

$$\begin{aligned} A &= 5 \cdot 3 + 6 \cdot 0 + 4 \cdot 1 + 2 \cdot 2 = 23, & B &= 5 \cdot 1 + 6 \cdot 2 + 4 \cdot 2 + 2 \cdot 3 = 31, \\ C &= 5 \cdot 0 + 6 \cdot 3 + 4 \cdot 0 + 2 \cdot 1 = 20, & D &= 5 \cdot 2 + 6 \cdot 1 + 4 \cdot 3 + 2 \cdot 0 = 28. \end{aligned}$$

Winner B (a check on the sum: $23+31+20+28 = 102 = 17 \cdot (3+2+1+0)$).

4. Pairwise contests (Condorcet). It suffices to go through the contests of candidate D :

$$D : A = 10 : 7, \quad D : B = 9 : 8, \quad D : C = 9 : 8.$$

(E.g. D vs. B : groups “4” and “5” vote for D , giving 9, and groups “6” and “2” vote for B , giving 8.) Candidate D beats everyone — it is the **Condorcet winner**, and it would therefore also win *sequential majority* voting under any bracket. The remaining contests give $B : A = 12 : 5$, $B : C = 11 : 6$, $A : C = 11 : 6$, so the tournament graph is acyclic and Slater’s ordering is $D > B > A > C$ (with no reversed edges).

What to take away from the example.

- The same preference schedule gives **four different winners**: plurality C , IRV A , Borda B , Condorcet/sequential D . “Who won the election” is therefore not a property of the voters, but of the choice of *rules*.
- The plurality winner C is here even the **Condorcet loser** — it loses *every* pairwise contest (11:6, 11:6, 9:8). This is the strongest possible illustration of why plurality does not satisfy the Condorcet winner condition.
- Neither Borda nor IRV elected the Condorcet winner D — neither of them satisfies the Condorcet winner condition. This is precisely why the Black / Baldwin / Nanson combinations were devised.
- Borda’s ordering ($B > D > A > C$) differs from the ordering by pairwise majority ($D > B > A > C$) — the difference between *consensual* and *majoritarian* voting.

The differing results for the same schedule are a concrete form of what Arrow's theorem says in general.

10.4 Properties of voting systems

Criteria

The Pareto property (unanimity): if *every* voter prefers X to Y , then $X >_{sw} Y$ should hold. *Satisfied by*: the majority system and the Borda count. *Not satisfied by*: sequential majority voting.

The Condorcet winner condition: a candidate who beats all the others in pairwise comparison (the *Condorcet winner*) should be the overall winner. *Satisfied by*: sequential majority voting and the Borda combinations (Black, Baldwin, Nanson). *Not satisfied by*: plurality, or the Borda count on its own.

Independence of irrelevant alternatives (IIA): whether $X >_{sw} Y$ should depend *only* on the relative ordering of X and Y in the voters' preferences. If the ordering of other candidates Z, W changes, the relation between X and Y must not be reversed. *Not satisfied by*: plurality, sequential majority voting, or the Borda count — practically no common system satisfies it.

Unrestricted domain (universality): the system must admit *arbitrary* preferences of *all* voters (and no other inputs), and for them deterministically return a *complete* ordering of the social choices.

Dictatorship: the result is determined by the preferences of a single voter. *Non-dictatorship*: there is no voter i such that whenever i strictly prefers x to y , x is socially chosen over y .

Theorem 10.1 (Arrow's impossibility theorem, 1951). *For three or more candidates there exists no preferential voting system that would convert individual ordered rankings into a complete and transitive social ordering and at the same time satisfy all of the following criteria:*

- *unrestricted domain,*
- *non-dictatorship,*
- *Pareto efficiency (unanimity),*
- *independence of irrelevant alternatives.*

A simpler (equivalent) formulation: for an election with more than two candidates, the only voting procedure satisfying the Pareto property and IIA is a *dictatorship*, in which the social outcome is simply given by one of the voters.

Interpretation. This is a *negative* result showing the fundamental limits of collective decision making. It certainly does *not* follow from it that the only workable system is a dictatorship — what follows is that every real voting system has to sacrifice some of the criteria (most often IIA) and that every system can be manipulated in some situation.

The Gibbard–Satterthwaite theorem (an addendum): every social choice function for three or more candidates that is *surjective* and *non-dictatorial* is **manipulable** — there is a situation in which it pays a voter to vote insincerely. This is fundamental for MAS: an agent is a computer, and tactical voting poses no moral problem for it. This is why MAS looks for mechanisms that are *strategy-proof*, which is the topic of the following chapter on auctions.

10.5 Exam summary

- Social welfare (a complete ordering) vs. social choice (a single winner); the ballot and the preference schedule.
- Systems: majority, plurality, plurality with elimination (IRV), sequential majority (the order decides), the Condorcet method, Borda ($N - 1, \dots, 0$), Slater (NP-hard), the Black / Baldwin / Nanson combinations.

- **The Condorcet paradox:** $A > B > C$, $B > C > A$, $C > A > B$ gives a cyclic collective preference; a Condorcet winner need not exist.
- Be able to work through a preference schedule (section 10.3): the same schedule gives plurality C , IRV A , Borda B , and Condorcet D — and the plurality winner may be the Condorcet *loser*.
- Criteria: Pareto (satisfied by majority and Borda, not by sequential), the Condorcet winner (satisfied by sequential and the Borda combinations), IIA (satisfied by almost nothing), universality, non-dictatorship.
- **Arrow's theorem:** for ≥ 3 candidates one cannot have universality, non-dictatorship, Pareto, and IIA at the same time. The simpler version: Pareto + IIA $\Rightarrow$ dictatorship.
- Gibbard–Satterthwaite: every reasonable voting system can be manipulated by insincere voting.

11 Resource allocation, auctions, and negotiation

11.1 Auctions as a mechanism of resource allocation

Auction

An auction is a **market institution** in which the messages from traders contain price information — an offer to buy at a given price (a *bid*), or an offer to sell at a given price (an *ask*) — and which gives priority to higher offers to buy and lower offers to sell.

In MAS an auction is above all a **mechanism for dividing scarce resources among agents**. *In the real world:* eBay, Sotheby's, mining licences, frequency bands for mobile operators. *In computer science:* allocating processor time, bandwidth, computational tasks, advertising slots.

The efficiency of an auction: it allocates the resource to the agent that wants it most (that has the highest valuation). The seller wants to *maximise* the price, the buyer to *minimise* it; each buyer has its own utility function.

Classification of auctions

By the valuation of the item:

- *private value* — the value depends only on the buyer (a concert ticket);
- *common value* — the value is the same for everybody, they just do not know it (mining rights) — hence the *winner's curse*: the winner is the one who erred the most in the upward direction;
- *correlated value* — a combination.

By protocol: the winner pays the first / the second price; *open cry* vs. *sealed bid*; a single round vs. several rounds.

11.2 Mechanism design

An auction is not merely “how bidding is done”; it is a special case of a more general problem. Game theory asks *how agents will behave under given rules*; **mechanism design** is “inverse game theory”: *which rules to choose so that rational selfish agents will do of their own accord what we want* — typically tell the truth.

A mechanism and its desirable properties

A **mechanism** is a pair: an *outcome selection rule*, which from the agents' declared preferences $\hat{v} = (\hat{v}_1, \dots, \hat{v}_n)$ selects an outcome $x(\hat{v})$, and a *payment rule* $p_i(\hat{v})$. The utility of an agent is $u_i = v_i(x(\hat{v})) - p_i(\hat{v})$ (*quasi-linear utility*). Desirable properties:

- **Incentive compatibility (IC)** — it pays the agent to declare the truth, $\hat{v}_i = v_i$. The strongest variant: truthfulness is a *dominant strategy* (*strategy-proof*, DSIC) — this is how Vickrey and VCG work.
- **Individual rationality** — nobody loses by participating ($u_i \geq 0$), otherwise the agents would not take part.

- **Efficiency** — the chosen outcome maximises social welfare $\sum_i v_i(x)$.
- **Budget balance** — the payments do not “evaporate” ($\sum_i p_i = 0$, or a non-negative revenue for the organiser).
- **Computational tractability** — both the outcome and the payments must be computable (typically a MAS concern, which economics is not interested in).

It is not always possible to have all these properties at once: the Myerson–Satterthwaite theorem says that for bilateral trade one cannot have IC, individual rationality, efficiency, and budget balance simultaneously. VCG sacrifices the budget, other mechanisms sacrifice efficiency.

The revelation principle

For every mechanism in which the agents play some (possibly very complicated) equilibrium strategies, there exists a **direct** mechanism in which the agents simply *declare their preferences truthfully* and the outcome is the same — the “lying” is built into the mechanism itself.

The consequence for us: in looking for a good mechanism it suffices to restrict attention to truthful direct mechanisms. This is why the theory revolves around Vickrey and VCG. *The limits:* the Gibbard–Satterthwaite theorem (chapter 10) shows that without money (with ordinal preferences only) a truthful non-dictatorial mechanism is impossible — it is only *payments* in a quasi-linear setting that allow Vickrey and VCG to circumvent this barrier.

11.3 The four basic types of auction

Auction	Protocol	Rational strategy	Notes
English	open cry, <i>ascending</i> price, the first (highest) price is paid	<i>dominant</i> : keep bidding up by ε until the price reaches my valuation, then withdraw	The most common one (antiques, art, the internet — about 95 % of auctions). The typical case in which the buyer overpays.
Dutch	open cry, <i>descending</i> price, the first to accept pays the current price	<i>no</i> dominant strategy exists; wait until the price falls just below my valuation — but this is a gamble	Fast; perishable goods (flowers, fish). Popular with sellers. It makes it impossible to estimate either the market price or the preferences of the others.
FPSB (<i>first-price sealed bid</i>)	a single round, sealed envelopes, the highest bid wins and pays its own price	<i>no</i> dominant strategy exists; bid <i>less</i> than one’s valuation (<i>bid shading</i>) — the optimum depends on an estimate of the others	Sales of real estate and government bonds. It does not force buyers to use their utility function and does not reveal the market price. <i>Strategically equivalent to the Dutch auction.</i>
Vickrey (<i>second-price sealed bid</i>)	a single round, sealed envelopes, the highest bid wins but pays the <i>second</i> highest	<i>dominant</i> : bid one’s valuation truthfully	Theoretically the most popular one; Google AdWords, stamp auctions. <i>Equivalent to the English auction</i> (with private values).

Why truthfulness is a dominant strategy in the Vickrey auction

Let my valuation be v , let me bid b , and let the highest bid of the others be p (which I do not know). I win exactly when $b > p$, and then I pay p ; my profit is $v - p$.

- **I bid $b > v$ (overstate):** compared with $b = v$ something changes only when $v < p < b$. Then I win, but I pay $p > v$ — a profit of $v - p < 0$. *This harms me.*
- **I bid $b < v$ (understate):** compared with $b = v$ something changes only when $b < p < v$. Then I lose, although I would have won with a profit of $v - p > 0$. *I forgo a profit.*

- In no case *can* I gain more by bidding $b \neq v$ than by bidding $b = v$.

Hence $b = v$ is a (weakly) dominant strategy — regardless of what the others do. The auction is **strategy-proof** (incentive compatible) and at the same time *efficient* (the bidder with the highest valuation wins).

Example — a comparison of revenues. Three buyers with valuations $A = 80$ CZK, $B = 60$ CZK, $C = 30$ CZK. In an ideal scenario (with no additional information and no cheating) the auctions turn out as follows:

Auction	Winner	Price
English	A	≈ 61 (bidding continues until B withdraws at 60)
Dutch	A	80 or slightly less (79) — A must not risk accepting too late
FPSB	A	80 (or $60 + \varepsilon$ if A knows the valuations of the others)
Vickrey	A	60 (the second highest bid)

In all cases the item is obtained by the agent with the highest valuation (the auctions are *efficient*), but they differ in the price and in the information the auction reveals.

Why does FPSB come out at 80 when one is supposed to bid below one's valuation? Because the scenario assumes *zero* information about the others: agent A does not know how high B will go, and any retreat from 80 risks losing. The degree of shading is a function of one's beliefs about the others — which is why FPSB (and the Dutch auction) **has no dominant strategy**. If A knows the valuations of the others, it bids $60 + \varepsilon$ and the seller's revenue drops to the level of the Vickrey auction (cf. the revenue equivalence theorem below). In the English and Vickrey auctions, by contrast, a dominant strategy does exist and does not depend on what the others do.

The revenue equivalence theorem

Vickrey (1961), Myerson, Riley–Samuelson. Under the assumptions: risk-neutral buyers, independent private values from the same continuous distribution, the item is obtained by the buyer with the highest valuation, and the buyer with the lowest possible valuation has zero expected profit — **all four basic auctions give the seller the same expected revenue**.

The differences in practice therefore stem from violations of the assumptions: risk aversion (in which case FPSB/Dutch are better), common values and the winner's curse (in which case the English auction is better because it reveals information), the possibility of collusion among buyers (the English auction is more vulnerable), and the cost of communication (sealed bid is cheaper).

11.4 Combinatorial auctions and VCG

Combinatorial auction

Several items are auctioned simultaneously and buyers submit bids on *subsets (bundles)*. The motivation: the values of items are not additive — *complementarity* (a left and a right shoe, two adjacent frequency blocks: $v(\{x, y\}) > v(x) + v(y)$) and *substitution* (two tickets for the same flight: $v(\{x, y\}) < v(x) + v(y)$).

The winner determination problem (WDP): select a set of mutually disjoint bids that maximises the sum of the prices. This is a generalisation of set packing — an **NP-hard** problem (and moreover hard to approximate). It is solved with ILP solvers and branch and bound (CABOB, CPLEX).

The second problem is *representational*: there are $2^m - 1$ possible bids on subsets; compact languages are used (OR, XOR, OR-of-XOR).

The VCG mechanism (Vickrey–Clarke–Groves)

A generalisation of the Vickrey auction to an arbitrary allocation. The principle:

1. Every agent i declares its valuation $\hat{v}_i(\cdot)$.
2. An allocation x^* maximising *social welfare* $\sum_i \hat{v}_i(x)$ is chosen.
3. Agent i pays the **externality it imposes on the others**:

$$p_i = \underbrace{\max_x \sum_{j \neq i} \hat{v}_j(x)}_{\text{the welfare of the others if } i \text{ did not exist}} - \underbrace{\sum_{j \neq i} \hat{v}_j(x^*)}_{\text{the welfare of the others under the chosen allocation}}.$$

Properties: truthfulness is a dominant strategy (*incentive compatible*), the allocation maximises social welfare (*efficient*), and the agents have a non-negative profit (*individually rational*). For a single item VCG reduces exactly to the Vickrey auction.

Disadvantages: computational cost (the WDP has to be solved $n + 1$ times), a possible zero or negative revenue for the seller, vulnerability to collusion and false identities (*shill bidding*), and the need to reveal one's true valuations (a privacy problem).

A VCG example. The items $\{x, y\}$ are auctioned. The bids: agent 1: $\{x\} \rightarrow 5$; agent 2: $\{y\} \rightarrow 4$; agent 3: $\{x, y\} \rightarrow 8$.

- The optimal allocation: $x \rightarrow$ agent 1, $y \rightarrow$ agent 2, total welfare $5 + 4 = 9 > 8$.
- The payment of agent 1: without agent 1 the best allocation would be “ $\{x, y\} \rightarrow$ agent 3” with value 8; under the allocation x^* the others have value 4 (agent 2 alone). Hence $p_1 = 8 - 4 = 4$. The profit of agent 1 is $5 - 4 = 1$.
- Symmetrically $p_2 = 8 - 5 = 3$, and the profit of agent 2 is $4 - 3 = 1$.
- The seller's total revenue is $4 + 3 = 7$, which is *less* than the 8 that agent 3 offered — a typical illustration that VCG does not maximise the seller's revenue.

11.5 Further types of auction and resource allocation

- An *all-pay auction* — everybody pays and the highest bid wins; a model of lobbying, bribes, sporting contests, and patent races.
- A *silent auction* — a written variant of the English one. The *Amsterdam auction* — it starts as an English one, and once two bidders remain it switches to a Dutch one with a doubled price. *Tsukiji* (the Tokyo fish market) — everybody bids at once, and ties are resolved by rock–paper–scissors.
- A *double auction* — bids are submitted by buyers and sellers alike and are matched; a model of a stock exchange.

Resource allocation in general (MARA)

Multiagent resource allocation: dividing a set of resources among agents according to their preferences. An auction is only one of the mechanisms (the *market* one); others are centralised optimisation, negotiation, or fair division. Criteria for the quality of an allocation:

- **utilitarian** — maximises $\sum_i u_i$ (social welfare);
- **egalitarian** — maximises $\min_i u_i$ (the worst-off agent);
- **Nash welfare** — maximises $\prod_i u_i$ (a compromise);
- **proportionality** — each of the n agents receives at least $1/n$ of its total value;
- **envy-freeness** — no agent prefers another agent's share to its own.

For divisible resources (“cake cutting”) there are classical protocols (*cut and choose* for two agents, *Dubins–Spanier* for n); for indivisible items both proportionality and envy-freeness are in general unattainable.

11.6 Negotiation

An auction requires a central auctioneer. The alternative is direct **negotiation** between agents. The classical model:

The monotonic concession protocol and the Zeuthen strategy

The **negotiation set** — the individually rational and Pareto-optimal deals.

The **monotonic concession protocol**: in each round both agents simultaneously propose a deal. It ends if one agent's proposal is at least as good for the other as that agent's own proposal; otherwise at least one of them must *concede* in the next round (propose a deal that is better for the counterpart), or else the negotiation breaks down.

The **Zeuthen strategy** answers the question of *who should concede*: the one who has the smaller "willingness to risk" a conflict,

$$\text{Risk}_i = \frac{u_i(\text{own proposal}) - u_i(\text{counterpart's proposal})}{u_i(\text{own proposal})},$$

that is, the ratio of what it loses by accepting the other's proposal to what it would lose by a conflict. The agent with the *lower* Risk concedes. It should concede just enough for the ratios to be reversed. A pair of Zeuthen strategies is a Nash equilibrium, and the resulting deal maximises the *Nash bargaining solution* (the product of the utilities).

Rosenschein and Zlotkin introduced for this type of negotiation the formalism of **task-oriented domains** (TOD): a pair of agents is assigned sets of tasks, a *deal* δ is a redistribution of the tasks between them, and the utility $u_i(\delta)$ is the cost the agent saves by the deal compared with performing its own tasks alone; the *conflict deal* Θ means that they fail to agree and each carries its own tasks itself. In TOD it may pay to **deceive**: an agent may declare *phantom* and *decoy* tasks that it does not have (in order to look more heavily loaded and concede less), or conversely *conceal* some of its tasks (*hidden tasks*); whether lying pays off depends on the structural properties of the domain. Further models of negotiation are **Rubinstein's alternating offers protocol**, in which the patience of the agents is modelled by a discount factor and in equilibrium the deal is struck right in the first round, and *argumentation-based negotiation*, in which the agents exchange not only proposals but also reasons (threats, promises, appeals).

11.7 Exam summary

- An auction = a mechanism for allocating scarce resources; an efficient auction gives the resource to whoever values it most. The dimensions: private/common value, open cry/sealed bid, first/second price, the number of rounds.
- **Mechanism design** = inverse game theory (I design the rules, not the strategy). The properties: incentive compatibility, individual rationality, efficiency, budget balance, computability — one cannot have everything at once. **The revelation principle**: it suffices to consider direct truthful mechanisms. Without payments it runs into Gibbard–Satterthwaite.
- The four basic types and their strategies: the English auction (bid up by ε up to your valuation) and Vickrey (bid truthfully) have a *dominant* strategy; the Dutch auction (wait for your valuation $-\varepsilon$) and FPSB (bid below your valuation) **do not**. Be able to give the **proof of truthfulness** for Vickrey.
- Equivalences: English $\leftrightarrow$ Vickrey, Dutch $\leftrightarrow$ FPSB. **The revenue equivalence theorem** and when it fails (risk aversion, common values, the winner's curse, collusion).
- Be able to do the example with valuations 80/60/30 and compute the price in all four auctions.
- Combinatorial auctions: bids on subsets (complementarity/substitution), the WDP is NP-hard. **VCG**: the allocation maximising social welfare + a payment equal to the externality the agent imposes on the others; truthful and efficient, but expensive and with a low revenue.
- Resource allocation: the utilitarian / egalitarian / Nash criterion, proportionality, envy-freeness.

- Negotiation: the monotonic concession protocol + the Zeuthen strategy (the one with the lower risk concedes); task-oriented domains and the three types of deception (phantom / decoy / hidden tasks); Rubinstein’s alternating offers with discounting.

12 Agent learning and reinforcement learning

12.1 Why and how agents learn

Intelligent agents in MAS should be *adaptive*. Learning is a change in an agent’s behaviour on the basis of experience; there is a whole range of methods:

Methods for agent learning

- **By the learning signal** (the three classical paradigms of machine learning): *supervised learning* (the agent receives the correct answers — e.g. learning a model of the opponent from observed moves, classifying percepts), *unsupervised learning* (finding structure in observations — clustering situations, discovering roles), and **reinforcement learning** (only a scalar reward from the environment is available; the most natural for autonomous agents, see the rest of this chapter).
- **Imitation learning** (*learning from demonstration*): the agent learns a policy from trajectories demonstrated by an expert — either by direct copying (*behavioural cloning*), or by *inverse RL*, in which the reward function is reconstructed from the expert’s behaviour.
- **Evolutionary methods**: a population of controllers (parameters of reactive behaviours, neural network weights, rules) is bred by a genetic algorithm according to the utility achieved — the typical way in which the reactive architectures of chapter 4 are “tuned experimentally”; this also includes *learning classifier systems* and neuroevolution.
- **By what the agent learns**: a model of the environment (the transition function), a model of the opponent (*opponent modelling*), a value function, the policy directly, or, in PRS, the ratings of plans.
- **Individual vs. social learning**: the agent learns only from its own experience, or also by imitating and sharing experience with the others (learning in a swarm, shared experience in MARL).

The most common and theoretically most developed approach to learning in MAS is **reinforcement learning** (RL) — a change of behaviour by trial and error on the basis of *rewards* from the environment. RL fits naturally into the formalism of chapter 2: the policy π is an action selection mechanism and the reward is an operationalisation of utility.

We distinguish:

- **Single-agent learning** — easier, and classical RL algorithms can be used (Q-learning).
- **Multiagent RL** (*MARL*) — the agents learn *simultaneously*; this is formalised by Markov games (Nash and Pareto optimality), and the state of the art is deep MARL (MADRL).

12.2 The Markov decision process

MDP — Markov Decision Process

An MDP is a quintuple $\langle S, A, T, R, \gamma \rangle$:

- S — the state space; A — the action space;
- $T : S \times A \times S \rightarrow [0, 1]$ — the transition function, $T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$;
- $R : S \times A \times S \rightarrow \mathbb{R}$ — the reward function (the immediate reward for a transition);
- $0 \leq \gamma < 1$ — the discount factor, a trade-off between immediate and future reward.

The **Markov property** holds: the next state and reward depend only on the current state and action, not on the whole history. (Compare this with the state transformer function τ :

$R^A \rightarrow 2^E$ from the abstract architecture, which *does* have a history — an MDP is its Markovian simplification enriched with probabilities and rewards.)

A **policy** $\pi : S \rightarrow A$ (or stochastically $\pi(a|s)$). The goal is to find an *optimal policy* π^* maximising the expected discounted sum of future rewards:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right].$$

The **value function** $V^\pi : S \rightarrow \mathbb{R}$ assigns to a state the expected utility if the agent follows the policy π ; the **action-value function** $Q^\pi(s, a)$ does the same for a state–action pair.

The Bellman equations

For a given policy (the *Bellman equation for evaluation*):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')].$$

For the optimal policy (the *Bellman optimality equation*):

$$\begin{aligned} V^*(s) &= \max_{a \in A} \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')], \\ Q^*(s, a) &= \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]. \end{aligned}$$

The optimal policy is obtained from Q^* as $\pi^*(s) = \arg \max_a Q^*(s, a)$.

Value iteration — if we know the complete description of the MDP:

- 1: initialise $V_0(s) \leftarrow 0$ for all s
- 2: **repeat**
- 3: **for all** $s \in S$ **do**
- 4: $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$
- 5: **end for**
- 6: **until** $\max_s |V_{k+1}(s) - V_k(s)| < \varepsilon$
- 7: **return** $\pi(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$

The Bellman operator is a contraction with coefficient γ , so the iteration converges to V^* geometrically fast. An alternative is **policy iteration**: alternately *evaluate* the current policy (solve a system of linear equations) and *improve* it (greedily according to V^π); it converges in finitely many steps, because there are finitely many policies.

12.3 Q-learning and SARSA

In reality we usually **do not know** either T or R . The agent therefore learns from experience by interacting with the environment in discrete steps — this is *model-free* RL.

Q-learning (Watkins, 1989)

The agent maintains a *table* $Q(s, a)$ — an estimate of the discounted sum of future rewards if action a is performed in state s . After every transition $s \xrightarrow{a} s'$ with reward r :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{the target (TD target)}} - Q(s, a) \right], \quad 0 \leq \alpha \leq 1 \text{ is the learning rate,}$$

equivalently $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$.

Off-policy: the target uses $\max_{a'}$, so it learns the value of the *optimal* policy regardless of the policy the agent actually acts by. Provided that every pair (s, a) is visited infinitely often and α decreases suitably, $Q \rightarrow Q^*$.

The exploration–exploitation dilemma; ε -greedy

The agent has to *exploit* what it already knows, but at the same time *explore* new actions. The standard solution:

$$\pi(s) = \begin{cases} \text{a random action from } A & \text{with probability } \varepsilon, \\ \arg \max_a Q(s, a) & \text{with probability } 1 - \varepsilon. \end{cases}$$

ε is typically decreased over time (*annealing*). Alternatives: softmax/Boltzmann $\pi(a|s) \propto e^{Q(s,a)/\tau}$, optimistic initialisation of Q , UCB (upper confidence bounds).

A concrete Q-learning step. Let $\alpha = 0.5$, $\gamma = 0.9$. The agent is in state s_1 , performs action a_1 , receives reward $r = 10$, and moves to s_2 . The current values: $Q(s_1, a_1) = 4$, $Q(s_2, a_1) = 6$, $Q(s_2, a_2) = 8$.

$$\max_{a'} Q(s_2, a') = \max(6, 8) = 8,$$

$$\text{TD target} = r + \gamma \max_{a'} Q(s_2, a') = 10 + 0.9 \cdot 8 = 17.2,$$

$$\text{TD error} = 17.2 - Q(s_1, a_1) = 17.2 - 4 = 13.2,$$

$$Q(s_1, a_1) \leftarrow 4 + 0.5 \cdot 13.2 = \mathbf{10.6}.$$

SARSA

State–Action–Reward–State–Action. The update uses the *actually chosen* next action a' (according to the same policy the agent acts by):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

On-policy: it learns the value of the policy the agent actually executes (including random exploration). The consequence: SARSA is *more cautious* — in the *cliff walking* task Q-learning learns the optimal path right at the edge of the precipice (and with ε -greedy it occasionally falls in), whereas SARSA learns a safer path further from the edge, because it factors in the risk of its own exploration.

A generalisation of both: TD(λ) with *eligibility traces*, which spread credit over several preceding steps.

12.4 Policy-based methods

Policy gradient, REINFORCE

Instead of value functions we optimise *the policy directly*. We parameterise the policy by a vector z , i.e. $\pi(a | s, z)$, and look for the optimal parameters z^* by gradient ascent on the expected return $J(z)$.

REINFORCE (Williams, 1992) — estimates the gradient from Monte Carlo simulations of whole episodes:

$$z_{t+1} = z_t + \alpha G_t \frac{\nabla_z \pi(A_t | S_t, z_t)}{\pi(A_t | S_t, z_t)} = z_t + \alpha G_t \nabla_z \ln \pi(A_t | S_t, z_t),$$

where G_t is the return from time t onwards. The intuition: increase the probability of actions that were followed by a high return.

Advantages over value-based methods: it handles *continuous* action spaces, naturally produces *stochastic* policies (necessary in multiagent games with mixed equilibria), and has better convergence properties under function approximation. **A disadvantage:** the high variance of the gradient estimate (addressed by subtracting a *baseline*).

Actor–critic

A combination of both approaches:

- The **actor** represents the *policy* — the action selection mechanism; it optimises it.
- The **critic** learns the *value function* — it estimates how good the actor’s performance is and thereby reduces the variance.

If the critic learns both $Q(s, a)$ and $V(s)$, the **advantage function** can be used:

$$A(s, a) = Q(s, a) - V(s),$$

i.e. the relative advantage of action a over the average — the policy gradient is then scaled by $A(s, a)$ instead of G_t , which substantially reduces the variance.

A3C (*Asynchronous Advantage Actor-Critic*) — several worker threads asynchronously compute gradients over their own copies of the environment and update shared parameters; the loss function consists of the *policy loss* for the actor and the *value loss* for the critic (plus an entropy regulariser encouraging exploration). More modern variants: A2C (synchronous), PPO, SAC, DDPG.

Deep learning. DQN (*Deep Q-Network*) replaces the Q table with a neural network (feed-forward, or recurrent for partially observable environments); the key tricks are *experience replay* (breaking the correlations between samples) and a *target network* (stabilising the target). In policy-based methods the policy is represented by a neural network (the input is the state, the output an action or its distribution, and the weights are the parameters z).

12.5 Multiagent reinforcement learning

Markov game (stochastic game)

A generalisation of the MDP to N agents: $\langle N, S, (A_i)_{i \in N}, T, (R_i)_{i \in N}, \gamma \rangle$.

- The **joint action space** is the product of the individual ones: $\mathbf{A} = A_1 \times \dots \times A_N$.
- $T : S \times \mathbf{A} \times S \rightarrow [0, 1]$ — the transition depends on the actions of *all* agents.
- Each agent has *its own* reward function R_i and its own policy π_i , but its value function V_i^π depends on the policies of *all* agents.

The connection with game theory: an agent’s policy is a *best response* to the joint policies of the others if it maximises its value function. The joint policies may be in a **Nash equilibrium** and may be **Pareto optimal**. Special cases: $N = 1$ gives an MDP; a game with a single state gives a normal-form game; $N = 2$ with zero sum gives a *minimax* game.

Minimax-Q and Nash-Q

Minimax-Q (Littman, 1994) — for two-agent zero-sum games. Instead of $\max_{a'} Q(s', a')$, the update uses the value of the *mixed minimax equilibrium* of the matrix $Q(s', \cdot, \cdot)$:

$$V(s') = \max_{\sigma \in \Delta(A_1)} \min_{a_2 \in A_2} \sum_{a_1} \sigma(a_1) Q(s', a_1, a_2),$$

which is a linear program. It converges to the minimax (and hence Nash) policy regardless of the opponent’s behaviour — at the price of *conservatism* (it will not exploit a weaker opponent).

Nash-Q (Hu, Wellman) — for general-sum games; the agent maintains Q_i for *all* agents and in the update uses the value of the Nash equilibrium of the stage game. It converges only under strong (and practically unverifiable) assumptions on the uniqueness of the equilibria. Related:

The problems of MARL

Game theory provides elegant formalisations (for few agents), but practice runs into the following:

- the environment is only **partially observable** (formally a Dec-POMDP, whose optimal solution is NEXP-complete);
- the state and action spaces may be **continuous**; the joint action space grows *exponentially* with the number of agents;
- the agents update their policies **simultaneously**, so from the point of view of a single agent the environment is **non-stationary** — the basic convergence assumption of RL is violated (“a moving target”);
- the agents may converge to **suboptimal** solutions or **oscillate** between several Nash equilibria;
- the **variance** of the value function estimates may be large;
- **credit assignment** — with a shared reward it is not clear which agent contributed.

The standard countermeasures: *centralised training with decentralised execution* (CTDE) — during training the critic sees the state and the actions of all agents, while at run time the actor sees only its own local observations (MADDPG, COMA, QMIX); furthermore *parameter sharing* between homogeneous agents and *opponent modelling*.

Self-play. The agent learns by playing against itself (or possibly against earlier versions of itself — *fictitious self-play*, leagues of opponents). In zero-sum games with perfect information, self-play reliably leads to strong strategies: TD-Gammon (Tesauro, 1992), AlphaGo Zero and AlphaZero (self-play + MCTS search + a deep network), AlphaStar (a league of opponents for StarCraft II). The risk: *cyclic* preferences among strategies (rock–paper–scissors), where naive self-play oscillates — hence leagues and population-based methods (*Policy-Space Response Oracles*).

LLM agents (a current trend). Large language models are used as an action selection mechanism (a policy), as a planner, as a critic in an actor-critic scheme, and for learning with a human in the loop. Typical applications in agent-based simulations: the spread of information in a graph of social interactions and synthetic electoral panels (LLM agents represent a population according to sociological surveys and estimate election results).

12.6 Exam summary

- Methods of agent learning: supervised / unsupervised / reinforcement; imitation learning and inverse RL; evolutionary methods (LCS, neuroevolution); what the agent learns (a model of the environment, a model of the opponent, a policy); individual vs. social learning. RL is the most common.
- **MDP** = $\langle S, A, T, R, \gamma \rangle$, the Markov property, a policy π , the value functions V^π, Q^π ; the Bellman optimality equation; value iteration (a contraction, converges) and policy iteration.
- **Q-learning**: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$, off-policy, tabular; be able to compute one step with numbers.
- **SARSA**: $\dots + \alpha[r + \gamma Q(s', a') - Q(s, a)]$, on-policy, more cautious (cliff walking). ϵ -greedy as the solution of the exploration/exploitation dilemma.
- **Policy gradient** (REINFORCE, $\nabla \ln \pi$ times the return) and **actor-critic** (the actor = the policy, the critic = the value function, the advantage $A = Q - V$); A3C; DQN (replay buffer, target network).
- **Markov game** = an MDP for N agents with a joint action space and individual rewards; best response, Nash equilibrium, Pareto optimality of the joint policies.
- **Minimax-Q** (zero-sum games, an LP in every update), Nash-Q (general sum, strong assumptions).
- The problems of MARL: non-stationarity, the exponential joint action space, partial observability,

credit assignment; CTDE as the solution. Self-play and AlphaZero.

13 Design methodologies, languages, and MAS platforms

13.1 Agent-oriented software engineering

Object-oriented methodologies (UML) are not sufficient, because agents are autonomous (their methods cannot be called directly), they have their own goals, and the relations between them are *organisational* (roles, protocols, commitments) rather than structural. This is why the methodologies of **AOSE** (agent-oriented software engineering) arose.

Gaia (Wooldridge, Jennings, Kinny, 2000)

One of the most cited AOSE methodologies. The key metaphor: a MAS is an **organisation** and agents play **roles**. It proceeds from requirements through analysis to design (leaving the implementation to the particular platform).

The analysis phase — two abstract models. The *roles model*: every role is described by four attributes — *permissions* (which resources it may read/modify), *responsibilities* (what it has to ensure; these split into *liveness* properties “something good happens”, described by a regular expression over activities and protocols, and *safety* properties “nothing bad happens”, described by invariants), *activities* (computations performed by the role itself), and *protocols* (interactions with other roles). The *interactions model*: the definition of the protocols between roles (purpose, initiator, responder, inputs, outputs).

The design phase — three concrete models: the *agent model* (the assignment of roles to agent types and the numbers of instances), the *services model* (the functions provided by a role — inputs, outputs, preconditions, effects), and the *acquaintance model* (a directed graph of who communicates with whom — a check for bottlenecks and excessive coupling).

Limitations: the original Gaia assumes a closed system with cooperative agents and a static organisation; the extensions *ROADMAP* and *Gaia v.2* relax these limitations.

Prometheus and Tropos

Prometheus (Padgham, Winikoff, 2002) — a practical methodology focused on **BDI agents**, with the PDT tool and the generation of code skeletons for JACK/Jason. Three phases: *system specification* (actors and scenarios, goals, *functionalities* as coherent units of behaviour, the interface with the surroundings — *percepts* and *actions*); *architectural design* (from the functionalities, *agent types* are derived according to cohesion and coupling, and protocols and messages are defined); *detailed design* (the inside of an agent: *capabilities*, plans, events, and data structures — directly the building blocks of BDI).

Tropos (Castro, Giorgini, Mylopoulos, 2002) — built on the **modelling of actors and goals** (the i^* framework); it is the only one that also covers the *early requirements* phase. Five phases: early requirements, late requirements, architectural design, detailed design, implementation. The concepts: *actor*, *goal* (and *softgoal* = a non-functional requirement with no sharp satisfaction criterion), *plan*, *resource*, and *dependency* (an actor depends on another one for a goal, a plan, or a resource). The analysis proceeds by means-ends decomposition of goals and by *contribution links* (+/−) to softgoals; the same concepts pervade the entire development from business analysis down to code.

Other methodologies (for orientation only): **MaSE** (from use cases to automata, with the agent-Tool tool), **INGENIAS** (a metamodel + code generation), **ADELFE** (adaptive/emergent systems), **O-MaSE**, **PASSI**.

13.2 The FIPA standard and the architecture of a platform

The FIPA reference architecture of an agent platform

- **AMS** (*Agent Management System*) — a mandatory component; the “registry” of the platform: it keeps records of agents, assigns them unique *AIDs* (agent identifiers), manages the life cycle (create, suspend, resume, terminate), and provides the *white pages*.
- **DF** (*Directory Facilitator*) — the *yellow pages*: agents register with it the *services* they offer and look up service providers.
- **MTS** (*Message Transport Service*) — the delivery of ACL messages within a platform and between platforms (the IIOP and HTTP protocols; bit-efficient, XML, and string encodings).
- **ACC** (*Agent Communication Channel*) — the interface to the MTS.

The standard further defines the ACL language, content languages (SL), interaction protocols, and ontology management.

13.3 Languages and platforms

The historical starting point is **agent-oriented programming** (AOP; Shoham 1993) — a paradigm in which the state of a program consists of *mental categories* (beliefs, *commitments*, and capabilities) and the program is a set of *commitment rules* which, on the basis of incoming messages and the current mental state, create new commitments. The first AOP language was Shoham’s **AGENT-0**: an agent = capabilities + initial beliefs + initial commitments + commitment rules; it communicates with three types of message (**request**, **unrequest**, **inform**). A different route is taken by **Concurrent MetateM** (Fisher): an agent is a *directly executable specification* in temporal logic — rules of the form “past $\Rightarrow$ future” are evaluated at run time and the agent acts so as to make its specification true. Both languages continue the line of deductive agents (chapter 3); the most successful representative of agent languages in practice today is AgentSpeak/Jason (see the table).

System	Type	Characteristics
JADE	Java, a FIPA platform	<i>Java Agent DEvelopment Framework</i> , one of the most widespread platforms for MAS, fully compliant with FIPA. An agent = a class inheriting from Agent with a setup() method; behaviour is composed of Behaviour objects (OneShot , Cyclic , Ticker , FSMBehaviour , SequentialBehaviour), which are scheduled by a cooperative (non-preemptive) scheduler inside the agent's single thread. Messages: ACLMessage , send() / blockingReceive() , MessageTemplate . Ready-made classes for protocols (ContractNetInitiator , AchieveREResponder), debugging tools (RMA, Sniffer, Introspector), containers, and agent mobility.
Jason	an AgentSpeak(L) interpreter in Java	The reference implementation of AgentSpeak , that is, of a BDI language (logic programming + BDI), a direct descendant of PRS. An agent program = initial <i>beliefs</i> , <i>rules</i> , and a <i>plan library</i> of the form trigger : context <- body . The trigger is an <i>event</i> (+ b the addition of a belief, +! g the addition of a goal, -! g the failure of a goal). The interpreter cycle: perceive → update the beliefs → select an event → find the <i>relevant</i> plans (by trigger) → select the <i>applicable</i> ones (by context) → create an intention → execute a step. Together with CARTAGO (the environment) and Moise (the organisation) it forms <i>JaCaMo</i> .
NetLogo	an environment for ABM	A language derived from Logo for <i>agent-based modelling</i> . The entities: <i>turtles</i> (mobile agents), <i>patches</i> (cells of the environment), <i>links</i> , and the <i>observer</i> . A model library (Segregation, Wolf Sheep Predation, Ants, Flocking) and <i>BehaviorSpace</i> for parametric studies. Intended for emergent behaviour and for teaching, not for FIPA communication.
SPADE	Python, XMPP	<i>Smart Python Agent Development Environment</i> . Communication via standard XMPP instead of a proprietary MTS; an agent = a class with <i>behaviours</i> (CyclicBehaviour , PeriodicBehaviour , FSMBehaviour), asyncio , and a web interface; popular thanks to its connection to the Python machine learning ecosystem.

Other environments: *Jadex*, *JACK*, and *JAM* (further implementations of BDI/PRS, with *Jadex* building a BDI layer on top of *JADE*); *MASON*, *Repast*, *GAMA*, *AnyLogic*, and *Mesa* (simulation platforms for agent-based models in sociology, economics, and epidemiology); *PettingZoo*, *Gymnasium*, *OpenSpiel*, and *SMAC* (standard environments for multiagent reinforcement learning); *LangGraph* and *AutoGen* (the orchestration of LLM agents).

13.4 Exam summary

- Why OO methodologies are not sufficient: autonomy, own goals, organisational (rather than structural) relations.
- **Gaia**: organisation + roles; analysis = the roles model (permissions, responsibilities with liveness and safety, activities, protocols) + the interactions model; design = the agent, services, and acquaintance models. The assumption of cooperative agents and a static organisation.
- **Prometheus**: system specification (goals, functionalities, percepts/actions) → architectural design (agent types, protocols, messages) → detailed design (capabilities, plans, events) — targeted at BDI.
- **Tropos**: actors, goals and softgoals, dependencies; it also covers early requirements (*i**).
- **The FIPA platform**: AMS (white pages, life cycle, AID), DF (yellow pages, services), MTS/ACC (the delivery of ACL messages).
- Agent-oriented programming (Shoham): the state = mental categories, the program = commit-

ment rules; AGENT-0; Concurrent MetateM = an executable temporal specification.

- **JADE** = Java, FIPA, **Agent** + **Behaviour** + **ACLMessage**, ready-made protocols and debugging tools. **Jason/AgentSpeak** = a BDI language, a plan of the form **trigger** : **context** <- **body**, a descendant of PRS. **NetLogo** = ABM simulation (turtles/patches/links). **SPADE** = Python + XMPP.

In closing: how to approach the exam

The topic is broad; the examiner usually checks that you understand the *connections*. There are four axes around which the questions revolve:

1. **Reactivity vs. proactiveness** — from the definition of an intelligent agent through commitment strategies (bold vs. cautious) and the parameters γ/ϕ in the Maes network all the way to the layers of hybrid architectures.
2. **Individual vs. collective rationality** — a selfish agent leads to (D, D) in the prisoner's dilemma, to the tragedy of the commons, and to tactical voting; hence the need for mechanism design (Vickrey, VCG), that is, for designing *rules* under which it pays a selfish agent to be truthful.
3. **Symbolic vs. subsymbolic representation** — FOL, STRIPS, ontologies, and ACL versus subsumption, activation networks, and neural networks in RL; practice is hybrid.
4. **Theory vs. practice** — elegant but non-constructive definitions (the optimal agent, Nash's theorem, Arrow's theorem) versus usable algorithms (A^* , CBS, Q-learning, CNET, JADE). Do not forget complexity: the PPAD-completeness of finding a NE, and the NP-hardness of the WDP, of optimal MAPF, and of Slater's system.

Small details that are often asked about: the difference between a *desire* and an *intention*; why the effect of **inform** is *not* that the recipient comes to believe; why (D, D) is the only *non*-Pareto-optimal outcome of the prisoner's dilemma; why truthfulness is dominant in the Vickrey auction; the difference between off-policy Q-learning and on-policy SARSA; and what CBS does at the high level and what at the low level.